What Keeps a Generative Recommender Relevant? An Empirical Study of Reinforcement Fine-Tuning Against a Learned Reward Model

Daniil Antonov

ITMO University, Saint Petersburg, Russia

Draft, version 2 12 October 2026

Abstract

Generative recommenders that produce a list item by item can be fine-tuned with reinforcement learning (RL) towards objectives that next-item training does not see. Published successes keep the reward tied to the log. We study the least protected setting, in which a learned model of user feedback is the only source of reward: a GPT-2 recommender is fine-tuned with REINFORCE and PPO to raise the consumed share of an item predicted by a second GPT-2, on a music log (Yambda-50M) and a video log (KION), in seven runs of one pipeline. A reward model fitted on disjoint users, relevance split by the logged outcome, the same lists scored for another user, and measures of list concentration separate real gains from exploitation. Without a constraint the policy always collapses to nearly one list for all users. With a KL anchor to the pretrained recommender the outcome depends on the dataset: on KION the predicted share rises by 23% in three runs, on Yambda by less than 2%. The gain is not personal: 99% of it is what other users would give the same lists, and re-ranking candidates by an item's mean share, with no reward model, reaches the same frontier. The anchor weight at which collapse begins depends on the advantage estimator and on how credit is assigned along the list. A logged-relevance reward keeps hits without an anchor; on KION it contracts the lists onto popular titles unless a weak anchor is added, on Yambda it raises NDCG@10 by 21% with almost no loss of coverage. A reward measured against another user cannot collapse onto one list, but buys little. Retention signals are predictable yet barely depend on the item shown. All results are offline and single-seed.

Keywords: generative recommendation, sequential recommendation, reinforcement learning, reward model, reward hacking, user retention, offline evaluation.

1 Introduction

Sequential recommenders built as autoregressive transformers over item or semantic identifiers [1], [2], [3], [4], [5] are trained to continue logged sequences. This objective imitates the log, and the log records what users did rather than what served them: in the two logs studied here, 36% of music listens stop before half of the track and 52% of video titles are abandoned before half of their length. A recommender that generates a list item by item is also a policy that takes a sequence of decisions, so reinforcement learning (RL) is the natural tool for moving it from "what comes next" towards "what will be received well" [6], [7], [8], [9].

Where the reward comes from decides how safe that step is. Two recent studies that report gains without losing relevance both keep the reward tied to the log. Petrov and Macdonald [8] fine-tune GPTRec with PPO on its own generated lists, with a reward that contains NDCG against the user's true next item next to a diversity or popularity term. Matveev et al. [9] train a retriever with off-policy REINFORCE on logged sessions, with logged feedback as the reward, and use a learned user feedback model only to evaluate. A third option is to let a learned model of user feedback *be* the reward. It is attractive because such a model scores any item for any user: the policy can be trained on its own generations and for feedback signals that the log holds only for the items users chose. It is also the option with the fewest protections. Model-based offline RL [10], [11], [12] and reward-model fine-tuning of language models [13], [14], [15] both document that a policy optimised against a learned model drifts to where the model is wrong, and in recommendation the drift is known to take the form of popularity and filter-bubble effects [16], [17], [18].

This paper studies the third option in its barest form. A GPT-2 [19] next-item recommender is fine-tuned with policy gradients to raise the share of an item that a second GPT-2, the *reward model*, predicts the user

will consume. Nothing in this reward asks for relevance; a KL penalty towards the pretrained recommender, which we call the *anchor*, is the only term that does. We ask five questions.

RQ1 Does fine-tuning raise the reward, and how does the behaviour of the recommender change?
RQ2 Is relevance preserved, do users still receive different lists, and which mechanism preserves either?
RQ3 Can longer-term signals, namely continuing the session and returning, serve as rewards?
RQ4 What happens along a list that is generated several steps ahead?
RQ5 What is the trade-off between reward and relevance, and does reaching it take RL?

The study covers two logs that differ in what makes the task hard (Yambda-50M: music, long histories with repeats; KION: video, short histories, one row per user and title), two estimators of the policy gradient (REINFORCE with a leave-one-out baseline, and PPO with a learned value model), and seven completed runs of the full pipeline. Because every reward gain is reported by a model, the pipeline carries instruments that do not depend on the model being optimised: a second reward model fitted on a disjoint half of the users that never influences training; relevance measured separately against future items by their logged outcome; the same lists scored for another user; and measures of how concentrated the lists are across users. Our findings are the following.

- **Unconstrained optimisation always collapses.** On both datasets and with both estimators, a policy without an anchor ends up serving nearly the same list to everyone. With a weak anchor on Yambda it moves to a handful of obscure tracks whose few listeners finished them: the optimised reward model reports +40%, the held-out one +16%, and NDCG@10 falls by 89%. The reward model was accurate and calibrated on held-out events (AUC 0.87); that did not protect it (Section 5.2).
- **An anchor that holds gives a dataset-specific result.** On KION the predicted share rises by 23%, the held-out model confirms it, and three runs agree. On Yambda the same weight under PPO changes the reward by less than 2% (Section 5.2).
- **The gain is not personal.** On KION 99% of the gain is what the reward model predicts for the same lists shown to *other* users, and re-ranking the pretrained recommender's candidates by the mean consumed share of an item, with no reward model at all, reaches the same frontier. The policy learns which items everybody finishes. The apparent confirmation by the log, relevance kept on finished items and lost on abandoned ones, largely disappears when the outcome is measured against what is usual for the item (Section 5.4, Section 5.5).
- **The anchor weight is not a transferable quantity.** On the same data the collapse begins between weights 0.1 and 0.3 under REINFORCE and between 0.03 and 0.1 under PPO, and removing the credit an item receives for the rest of the list moves the policy up to thirty times further at one and the same weight (Section 5.3, Section 5.9).
- **Each place for the relevance requirement protects something different.** The anchor preserves the distribution of the pretrained recommender. A logged-relevance reward [8] preserves hits with no anchor at all; what is easiest to hit decides the rest. On KION that is popular titles, and coverage falls from 0.50 to 0.10 unless a weak anchor is added, in which case the reward rises by 20% while NDCG@10 rises by 9% on finished items and falls by 2% overall. On Yambda it is the user's own earlier tracks, and NDCG@10 rises by 21% with almost no loss of coverage. Learning only from logged actions [9] preserves both hits and variety and gains less reward. A choice head inside the reward model fails as a relevance signal (Section 5.6, Section 5.9).
- **A reward measured against another user cannot collapse onto one list, and finds little to gain.** Subtracting what the reward model predicts for the same list shown to someone else keeps the lists of different users apart even with no anchor, but the part of the predicted share that a policy can make personal is a few points at most (Section 5.5).
- **Retention signals are predictable but offer little leverage.** Returning is predicted with AUC 0.70–0.72 on KION and 0.60 on Yambda, yet the signal varies four to nine times less across one user's candidates than across users; rewarding it buys little on KION and nothing on Yambda (Section 5.7).
- **A moderate gain does not need RL.** Re-ranking the pretrained recommender's 100 best candidates is at least as relevant as RL up to a reward gain of about one third on KION, supervised fine-tuning on liked items and off-policy REINFORCE land on the main RL run, and the multi-step predicted reward that only RL optimises depends by 25 to 35 points on an assumption of the simulation (Section 5.8, Section 5.9).

All results are offline, and every configuration was trained once. Section 7 lists what this leaves open.

2 Related Work

Generative and sequential recommenders. The recurrent GRU4Rec [20] and the self-attentive SASRec [1] and BERT4Rec [21] made next-item prediction the default formulation of sequential recommendation. Generative recommenders use the same models autoregressively: GPTRec [2] generates a list item by item (Next-K) instead of scoring the catalogue once (Top-K), TIGER [3] generates semantic identifiers, and industrial systems scale the idea to whole retrieval and ranking stacks [4], [5], [22]. We keep the simplest member of the family, a GPT-2 decoder over item-identifier tokens, so that the effects of fine-tuning are not confounded with tokenisation.

RL for recommendation. Policy-gradient recommenders trained from logs correct for the logging policy with importance weights [6]; value-based methods decompose the value of a slate [7] or act in a learned slate space [23]; SQN and SA2C attach RL heads to a supervised sequential model [24], [25]. The two studies we build on sit at opposite ends: on-policy PPO over the recommender's own generations with a reward read from the true next item [8], and off-policy REINFORCE with a multi-step correction over logged sessions [9]. Learned models enter rewards when the wanted signal is sparse, as with imputed satisfaction responses [26], and they replace the user altogether in simulators [27], [28], [29]. Our setting is the second case with a simulator learned from the same log the recommender was trained on.

Learned rewards and their exploitation. Model-based offline RL treats the problem as one of trusting a model only where data supports it: MOPO penalises the reward by model uncertainty [10], MOReL builds a pessimistic MDP [11], and MBPO keeps model rollouts short [12]. For language models fine-tuned against learned reward models, Gao et al. [13] show that a "gold" reward first rises and then falls while the proxy keeps rising as the policy moves away from its initialisation, Coste et al. [14] mitigate this with ensembles, and Skalse et al. [15] formalise reward hacking. Our held-out reward model plays the part of the gold model at no extra scale: same architecture, other users. In recommendation, conservative offline RL has been shown to amplify the Matthew effect [16], filter bubbles have been studied as a long-term effect of interactive recommendation [17], and feedback loops have been shown to degenerate [18].

Staying close to a reference policy. TRPO and PPO bound each update relative to the previous iterate [30], [31]. Penalising divergence from a *pretrained* model is the older idea of KL-control [32], [33], used in fine-tuning language models from human preferences [34], [35] and, as behaviour regularisation, in offline RL [36]. Our anchor belongs to this line. What we add is evidence that its weight has no meaning across advantage estimators.

Long-term signals and consumption-time rewards. RL has been applied to long-term engagement [37] and directly to retention [38]; delayed outcomes are commonly approached through proxies [39], and measuring the long-term value of exploration takes dedicated experiment designs [40]. Consumption time is confounded by item duration [41], [42]. We observe the same confounding from the policy's side: the policy moves along the duration axis in whichever direction the normalisation of the reward favours.

Multi-step generation. Feeding a model its own outputs takes it away from the inputs it was trained on [43], [44]; Petrov and Macdonald report that Next-K generation is worse than Top-K ranking after supervised training [8]. Spending inference compute on several candidates that a learned model judges is the recommender form of test-time scaling [9], [45].

Evaluation. Next-item protocols hide deficiencies of RL recommenders [46], and off-policy estimators such as the doubly robust one [47] need logged propensities, which neither of our logs provides. We therefore rely on a held-out model and on model-free checks, and repeat the caveat of [9] that results selected and judged by the same feedback model are not independent evidence. The baselines are trained under one protocol, following the concerns of [48].

3 Method

Figure 1 shows the pipeline: a recommender is pretrained on next-item prediction, two reward models are fitted on disjoint halves of the users, and the recommender is fine-tuned on the reward that one of them predicts for its own generated lists.

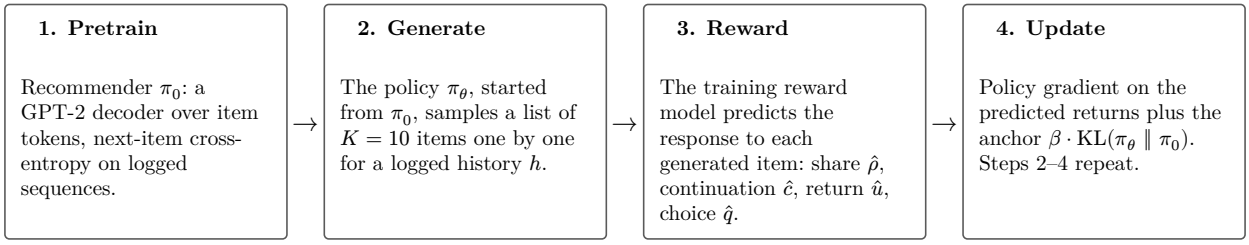


Figure 1: The pipeline. The reward model in step 3 is the only source of reward during fine-tuning; the instruments in the dashed box never influence the policy.

3.1 Setting and notation

A log is a set of events, each with a user, an item i_t, a time and the *consumed share* $\rho_t \in [0, 1]$: the share of the track that was played or of the title that was watched. A recommender is a distribution $\pi_\theta(j \mid h)$ over the next item given the history h; π_0 is the recommender before fine-tuning. A recommendation is a list $a_{1:K}$ with $K = 10$. For evaluation, F is the set of the next $M = 20$ items the user really consumed after h. It is split in two ways: into *liked* items with $\rho \geq 0.5$ and *skipped* items, and into items consumed *above usual*, $\rho \geq \bar{\rho}(i)$, and *below usual*, where $\bar{\rho}(i)$ is the mean consumed share of item i in the training period.

3.2 Base recommender

The recommender is a GPT-2 decoder [19] over item tokens with tied input and output embeddings. Its state s_t at position t summarises $i_{1:t}$, and $\pi_\theta(j \mid i_{1:t}) = \mathrm{softmax}_j(Es_t)$ with the embedding matrix E. It is trained with the full-softmax cross-entropy

$$\mathcal{L}_{\text{next}}(\theta) = -\sum_t \log \pi_\theta(i_{t+1} \mid i_{1:t}). \tag{1}$$

A list is obtained in one of two ways [2]. *Ranking* (Top-K) scores the catalogue once and takes the K best items. *Generation* (Next-K) appends each chosen item to the history as if the user had consumed it, $a_k = \arg\max_j \pi_\theta(j \mid h, a_{1:k-1})$, and never repeats an item within the list. Fine-tuning changes the second mode, in which the recommender is a policy over a sequence of decisions.

3.3 Reward model

The reward model is a second GPT-2 decoder of the same size. The token at position t carries the item i_t together with the consumed share of the *previous* item, ρ_{t-1}, in ten buckets. The state at position t has therefore seen the candidate item and the whole history with its responses, but not the response it has to predict. One forward pass scores every event of a real history, and a generated item can be scored before any outcome for it exists. Four heads read this state.

- $\hat{\rho}$, the consumed share, trained with binary cross-entropy against ρ as a soft label.
- $\hat{c}$, the probability that the session continues: the user's next event follows within a gap g.
- $\hat{u}$, the probability that the user returns: a new session starts within a window w.
- $\hat{q}$, the probability that the user would choose the item at all.

The loss is the sum of the four terms, $\mathcal{L} = \mathrm{BCE}(\hat{\rho}, \rho) + \lambda_c \,\mathrm{BCE}(\hat{c}, c) + \lambda_u \,\mathrm{BCE}(\hat{u}, u) + \lambda_q \mathcal{L}_q$. The first two heads are present in all runs; the last two were added from run E on (Section 4).

Retention labels. For an event at time τ and later events of the same user at times τ',

$$c = \mathbb{1}\Big[\min_{\tau'}(\tau' - \tau) \leq g\Big], \quad u = \mathbb{1}[\exists \tau' : g < \tau' - \tau \leq w]. \tag{2}$$

When nothing follows an event, either the user did not come back or the log ended before the window closed. We treat an outcome as known only if the whole waiting time lies inside the observed data, $\tau + w \leq T$ for a log that ends at T; unknown outcomes take part in neither training nor evaluation.

Choice head. The log holds only items that users picked, so $\hat{\rho}$ estimates the share *given that the user is consuming the item* and says nothing about an item this user would never pick. To give the reward model

a notion of choice, one item per training window is replaced by the item of another event in the batch, and $\hat{q}$ is trained to tell logged items from replacements:

$$\mathcal{L}_q = -\frac{1}{2}\mathbb{E}[\log \hat{q}(i_t \mid h_t)] - \frac{1}{2}\mathbb{E}_{j\sim p}[\log(1 - \hat{q}(j \mid h_t))]. \tag{3}$$

Replacements follow the popularity distribution p, so popularity alone cannot separate them. As in noise-contrastive estimation [49], the optimum is $\hat{q}^*(j \mid h) = P(j \mid h)/(P(j \mid h) + p(j))$: above one half when the item is more likely for this user than on average.

Two copies. Two reward models are trained on disjoint halves of the users. The *training* reward model produces the rewards during fine-tuning. The *held-out* reward model never influences the policy and only scores its lists. A gain that the held-out model does not confirm is exploitation of the errors of the training model.

3.4 Reinforcement fine-tuning

One episode is the generation of one list. The state is $s_k = (h, a_{1:k-1})$, the action is the next item a_k, the reward r_k is predicted by the training reward model (by default $r_k = \hat{\rho}_k$), and the return is $G_k = \sum_{l\geq k} \gamma^{l-k} r_l$. No real user is in the loop: the reward model is the environment. To score the k-th item it needs the responses to the items generated before it, which nobody observed; by default it is fed its own expectation $\hat{\rho}$ for them. Section 5.8 examines this assumption.

Histories are sampled from the training period as a user and a cut-off point. The policy gradient is estimated as $\mathbb{E}\left[\sum_k A_k \nabla_\theta \log \pi_\theta(a_k \mid s_k)\right]$ with one of two advantage estimators.

REINFORCE with a leave-one-out baseline. Several lists are sampled for each history, and the baseline of a list is the mean return of the other lists sampled for the same history [50], [51], which leaves the gradient unbiased and removes the part of the return that depends on the user alone.

PPO with a learned value model. A value model V_φ, a copy of the recommender's transformer with a scalar head, predicts the return from every position. Advantages are generalised advantage estimates [52], $A_k = \sum_{l\geq 0} (\gamma\lambda)^l \delta_{k+l}$ with $\delta_k = r_k + \gamma V_\varphi(s_{k+1}) - V_\varphi(s_k)$, and each batch of lists is used for several steps on the clipped surrogate [31]

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}[\min(w_k A_k, \text{clip}(w_k, 1-\varepsilon, 1+\varepsilon) A_k)], \quad w_k = \frac{\pi_\theta(a_k \mid s_k)}{\pi_{\theta_{\text{old}}}(a_k \mid s_k)}. \tag{4}$$

In both cases advantages are scaled to zero mean and unit variance within the batch, and dropout is switched off during fine-tuning so that recorded and recomputed log-probabilities agree.

Anchor. The full loss adds a penalty on the divergence from the frozen pretrained recommender, computed exactly over the catalogue at every position:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PG}}(\theta) + \beta \cdot \mathbb{E}_k[\text{KL}(\pi_\theta(\cdot \mid s_k) \parallel \pi_0(\cdot \mid s_k))], \tag{5}$$

where $\mathcal{L}_{\text{PG}}$ is the REINFORCE or the PPO term. Since the default reward has no relevance term, the weight β is the only thing that asks the policy to stay relevant.

Credit along the list. Two ablations test whether fine-tuning uses the sequential structure of a list. With $\gamma = 0$ the return reduces to the reward of the item itself, $G_k = r_k$, so no item is credited with what follows it. With *one-item episodes* a single item is sampled per episode during fine-tuning, so the policy never learns from a context that contains generated items; at evaluation its lists are still generated item by item.

3.5 Reward family

All rewards studied here are instances of

$$r_k = \hat{q}_k^{\chi}\left[w_\rho(\hat{\rho}_k - b_k) + w_c \hat{c}_k + w_u \hat{u}_k\right] + w_F \frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)}, \quad \chi \in \{0, 1\}. \tag{6}$$

The default is $w_\rho = 1$ with everything else at zero. The *time* variant multiplies $\hat{\rho}_k$ by the duration of the item relative to the median duration in the catalogue.

Relevance inside the reward. The last term is the *logged-relevance* reward of Petrov and Macdonald [8]: for a history sampled from the training period the user's real continuation F is known, and the sum of this term over the list is DCG@K. With $\chi = 1$ the model-predicted part is weighted by the choice probability,

so that an item the user would not pick earns little however high its average consumed share. Both put the relevance requirement *inside* the reward, where the anchor puts it outside.

Personal rewards. Without a baseline b_k, the reward pays the most for items that everybody finishes, whoever receives them. Two baselines pay only for what depends on the user. The *personal share* subtracts what the same reward model predicts for the same list shown to another user with history h', $b_k = \hat{\rho}(a_k \mid h', a_{1:k-1})$. For a policy that ignores its user this reward has zero expectation, so a list that is the same for everyone earns nothing, and every error of the reward model that depends on the item alone cancels. The *share above the item mean* subtracts the training-period mean of the item, $b_k = \bar{\rho}(a_k)$.

3.6 Alternatives to on-policy RL

Four methods raise the same reward without generating lists in a training loop.

- **Liked-only fine-tuning** continues supervised next-item training for one epoch, on targets with $\rho \geq 0.5$ only. It uses logged feedback and no reward model.
- **Off-policy REINFORCE** on logged sessions, after [9]. Nothing is generated and no reward model is asked: the log-probability of every logged item is raised in proportion to the logged reward that followed it,

$$\mathcal{L}(\theta) = -\frac{\sum_t \gamma^{n_t} \bar{\omega}_t G_t \log \pi_\theta(i_t \mid i_{1:t-1})}{\sum_t \gamma^{n_t} \bar{\omega}_t}, \quad \bar{\omega}_t = \min\left(\kappa, \prod_{l=t}^{t+L-1} \frac{\pi_\theta(i_l \mid i_{1:l-1})}{\pi_0(i_l \mid i_{1:l-1})}\right), \tag{7}$$

  where G_t is the discounted sum of logged rewards to the end of the session, n_t the position of the action in its session, and the frozen pretrained recommender stands for the behaviour policy. The cap κ on the importance weights limits how far the policy may move. We use $\gamma = 0.9$ and $L = 10$. Unlike the original, every logged event counts as an action, because KION does not mark which events a recommender served.
- **Re-ranking** takes the 100 best candidates of the pretrained recommender and keeps the K with the highest $r(j \mid h) + w \log \pi_0(j \mid h)$, where r is the reward predicted for j as the next item after the real history. Nothing is trained; the weight w plays the part of the anchor.
- **Best of n sampled lists** samples n lists from the pretrained recommender and serves the one with the highest predicted return, the test-time scaling of [9] applied to whole lists.

3.7 Whose reward is it

A list generated for a user with history h can also be shown to another user with history h'. The reward model then gives two values, $R_{\text{own}} = \mathbb{E}[\hat{\rho}(a \mid h)]$ and $R_{\text{other}} = \mathbb{E}[\hat{\rho}(a \mid h')]$. Their difference is the part of the predicted response that depends on who receives the list, the *personal part*; it is zero for any recommender that ignores the user. The gain of a method over π_0 splits accordingly,

$$\Delta R_{\text{own}} = \Delta R_{\text{other}} + \Delta(R_{\text{own}} - R_{\text{other}}), \tag{8}$$

into the gain that other users would give the same lists and the rest, whose share of the total we call the *personal share of the gain*. As a reference that uses no reward model at all, the candidates of π_0 are also re-ranked by $\bar{\rho}(j) + w \log \pi_0(j \mid h)$, the mean consumed share of the item in the training period.

4 Experimental Setup

4.1 Data

Yambda-50M [53] is a listening log of Yandex Music: 46.5 million listens of 10,000 users over about 300 days. Each event has the track, a timestamp, the played share of the track, the track length and a flag telling whether the user chose the track or a recommender served it. **KION** [54] is a log of the MTS online cinema: 5.5 million rows of 962 thousand users over about five months. It has one row per user and title, with the date of the last viewing, the total watch time and the watched share of the title. Time is given as dates, and nothing says whether a title was recommended.

Both logs are split along one time axis: the last 7 days are the test period, the 7 days before them the validation period, the rest the training period. The catalogue, popularity and all item statistics come from the training period only. On Yambda the 50,000 most popular training-period tracks become tokens,

Table 1: The two processed logs. Session and return are defined in the text.

	Yambda-50M	**KION**
Events kept	37.2 M (80.1%)	3.88 M (70.9%)
Users	8,711	256,407
Items in the catalogue	50,000	7,992
Events per user, median	2,638	10
Training / validation / test events	35.0 M / 1.13 M / 1.08 M	3.51 M / 0.19 M / 0.18 M
Mean consumed share	0.65	0.50
Events consumed at least half	64%	48%
Events that continue the session	95%	42%
Events followed by a return	92%	71%
Test users with a request	7,593	59,997

numbered by popularity, events on other tracks are dropped, and a user needs at least 50 training events. On KION a title needs 10 training events and a user 5. Table 1 compares the processed logs.

The consumed share is bimodal in both logs (Figure 12 in the appendix): an item is either dropped early or finished, so the reward model is in effect a skip classifier. The two logs differ in how the share depends on the length of the item. On Yambda short tracks are finished somewhat more often than long ones, 0.70 against 0.60 between the extreme duration deciles. On KION titles of film length, roughly 80 to 140 minutes, are watched to 0.5 on average, while the longest tenth of the catalogue, more than 8.5 hours of content and hence series, is watched to 0.22, because the share is taken of the whole series. A reward built on this share prefers films before any model is involved.

Retention signals. On Yambda the session continues if the same user has another event within 30 minutes, and the user returns by starting a new session within a day. KION only has dates: a session is one calendar day, and the user returns by watching on another day within a week. The outcome of return is known for 99% of Yambda events and 89% of KION events. The order of a KION user's titles within a day is the order of rows in the file, so on KION session continuation says how many titles the day had more than which title kept the viewer. Runs A to D have a single signal, "continues the session", with the 30-minute gap on Yambda and a 7-day gap on KION.

4.2 Models and training

The recommender has 4 layers, width 256 and 4 attention heads. On Yambda it is trained on windows of 128 consecutive events with dropout 0.1; on KION on windows of 48 events with dropout 0.2. Optimisation uses AdamW [55] with learning rate 10^{-3}, cosine decay and early stopping on the validation loss. Re-recommending items from the history is allowed on Yambda, where repeated listening is normal, and blocked on KION, where every user–title pair occurs once; baselines and re-ranking obey the same constraints. The reward models have the same size.

REINFORCE (runs A and B) samples 4 lists per history and uses a learning rate of $3 \cdot 10^{-5}$. PPO (runs C to G) samples one list for each of 512 histories per iteration and takes four gradient steps on them with clipping threshold $\varepsilon = 0.2$, discount $\gamma = 0.99$ and $\lambda = 0.9$. The learning rates are 10^{-5} for the policy and 10^{-4} for the value model, which is trained alone for the first 50 iterations.

4.3 Evaluation

Each test user with a history makes one request at the start of the test week. The last 118 events (38 on KION) are the history and the next at most 20 events are the ground truth. This gives 7,593 requests on Yambda and 59,997 on KION. Four groups of quantities are measured on lists of ten.

- **Relevance.** Recall, NDCG, MAP and HitRate at 1, 5 and 10 against all future items and against each part of the two splits. Neither split needs a reward model. A policy that has really learned to avoid what users abandon should lose relevance on skipped items and keep it on liked ones. The second split asks the same question without favouring items that are easy to finish: an item counts as consumed *above usual* only if this user consumed more of it than its viewers do on average.
- **Predicted reward.** Both reward models score the generated lists.
- **Personal part.** The held-out reward model scores every list for its own user and for another user (Section 3.7).

- **Concentration.** Catalogue coverage (the share of items that appear in at least one list), the share of users who receive the most frequent item (*top-item share*), the expected overlap between the lists of two users (*pair overlap*), the share of recommendations already in the history, and the popularity and duration of recommended items.

In the PPO runs the change from the pretrained to the fine-tuned recommender is reported with a 95% interval from a paired bootstrap over users. It measures the noise of the user sample, not the noise between training runs.

4.4 Baselines

The pretrained recommender is compared with the most popular items, a replay of the user's own history (most played first, the rest by popularity), a first-order Markov chain, GRU4Rec with the full softmax [20], SASRec with its original loss (binary cross-entropy against one sampled negative per position) [1] and BERT4Rec trained on the cloze task [21]. The neural baselines take the width, depth, dropout and window length of the recommender, are trained on the same windows and are stopped by their own validation loss. None of them was tuned.

4.5 Runs

The pipeline was completed seven times (Table 15 in the appendix). Runs A and B use REINFORCE on Yambda; B is a reduced-budget version on a much weaker GPU with a 30,000-track catalogue. Runs C and D use PPO on KION and on Yambda and are the first with baselines, alternatives to RL and intervals. Run E repeats run C with reward models that have the return and choice heads and adds rewards for retention and for relevance. Runs F (Yambda) and G (KION) use the current pipeline: they repeat the PPO runs and add the replay baseline, the second relevance split, the personal part, personal rewards and the credit ablations. The anchor weight β is swept in every run, and the *main* RL run of each uses a fixed weight: 0.1 in runs A and B, 0.3 in runs C to G.

Unless a run is named, the main text reports run A for REINFORCE on Yambda, run F for PPO on Yambda and run G for PPO on KION. Experiments that exist in an earlier run only are labelled with that run. Runs C, E and G share one recommender on KION and runs D and F one on Yambda, so the repeated PPO runs measure the variation that comes from the reward models and from fine-tuning, not from pretraining (Table 16 in the appendix).

5 Results

5.1 The pretrained models are sound, with one caveat

Recommender. Table 2 places the pretrained recommender among the baselines. On Yambda it is the best of the trained models: 4% above GRU4Rec, 8% above BERT4Rec and 48% above SASRec in NDCG@10 for ranked lists. Two caveats apply to that ranking: GRU4Rec and BERT4Rec reached the limit of 10 epochs while their validation loss was still falling, and no intervals were computed between baselines. The NDCG@10 of SASRec, 0.077, is close to the 0.075 published for Yambda-50M [53], although the protocols differ.

The caveat in the heading is the replay baseline. A list made only of the user's own past tracks, most played first, reaches NDCG@10 0.187 on Yambda, against 0.114 to 0.117 for the recommender and less for every other trained model. Repeated listening dominates this log, and a next-item model spends half of its list on tracks the user has not played (52% of its generated items come from the history). Every statement below about relevance on Yambda is relative to this recommender, and Section 5.6 returns to the gap.

On KION popularity is a strong floor: the ten most popular unseen titles reach NDCG@10 0.090, and the recommender is 35% above it, against a factor of six on Yambda. The competition for which the dataset was released shows the same picture: its five best solutions scored MAP@10 between 0.114 and 0.122 against 0.091 for popularity [54][1], a ratio of 1.25 to 1.34; ours is 1.37 for the GPT recommender and 1.46 for BERT4Rec. Protocols differ, so only the ratio is comparable. The recommender is mid-table on KION: level with GRU4Rec, 13% above SASRec and 5% below BERT4Rec. Lists are concentrated on KION before any

[1]Competition results are listed in the README of `github.com/irsafilo/KION_DATASET`.

Table 2: Baselines and the GPT recommender on lists of ten (runs F and G; runs D and C gave the same numbers). Every model is trained once and none is tuned. On KION seen titles are never recommended, so replaying the history reduces to popularity.

	Popular	**Replay**	**Markov**	**GRU4Rec**	**SASRec**	**BERT4Rec**	**GPT, ranked**	**GPT, generated**
Yambda-50M, 7,593 test users								
NDCG@1	0.0246	0.2509	0.0979	0.1488	0.1030	0.1534	0.1653	0.1653
NDCG@10	0.0186	0.1873	0.0550	0.1097	0.0767	0.1054	0.1136	0.1165
Recall@10	0.0114	0.1238	0.0335	0.0718	0.0506	0.0673	0.0737	0.0742
MAP@10	0.0079	0.1294	0.0272	0.0591	0.0366	0.0585	0.0621	0.0747
HitRate@10	0.1118	0.4599	0.2612	0.4417	0.3806	0.4163	0.4454	0.3656
Catalogue coverage	0.0002	0.519	0.457	0.448	0.397	0.459	0.464	0.519
Share from the history	0.083	0.999	0.186	0.426	0.325	0.438	0.423	0.520
KION, 59,997 test users								
NDCG@1	0.0597	0.0597	0.0669	0.0877	0.0726	0.0945	0.0885	0.0885
NDCG@10	0.0903	0.0903	0.0959	0.1215	0.1082	0.1290	0.1220	0.1173
Recall@10	0.1303	0.1303	0.1394	0.1721	0.1580	0.1811	0.1726	0.1620
MAP@10	0.0607	0.0607	0.0637	0.0824	0.0720	0.0885	0.0831	0.0819
HitRate@10	0.2358	0.2358	0.2571	0.3154	0.2903	0.3281	0.3144	0.2871
Catalogue coverage	0.004	0.004	0.466	0.544	0.335	0.427	0.574	0.495
Top-item share	0.918	0.918	0.530	0.502	0.504	0.496	0.484	0.566
Pair overlap	0.714	0.714	0.220	0.149	0.161	0.161	0.140	0.209

RL: every neural model gives its most frequent title to about half of the users, and two users share 14% to 21% of their lists, against 2% and 0.1% on Yambda. Concentration measures therefore start high on KION.

Ranking against generation. Generating the list item by item does not hurt on Yambda (NDCG@10 0.117 against 0.114 for ranking) and hurts slightly on KION (0.117 against 0.122, with a smaller coverage). The KION result is the one Petrov and Macdonald report [8]. A likely reason for the difference is that Yambda users replay tracks, so a generated track is a plausible part of the history it joins, while a KION title is watched once; in addition our ground truth is the next 20 events and not the single next item.

Reward models. Both reward models predict held-out feedback much better than simple predictors (Table 3). On Yambda they are almost perfectly calibrated. On KION calibration is good in the middle of the range and compressed at the ends: in run C, events predicted at about 0.06 are watched to 0.12 and those predicted at about 0.93 to 0.87. The two copies agree, with a correlation between their predictions on test events of 0.95 on Yambda and 0.93 on KION. Three properties matter for what follows.

- *Where the skill comes from.* On Yambda the recent behaviour of the user alone reaches AUC 0.80 and the mean share of the track only 0.61: most of the model's skill is knowing whether this user is skipping right now, and knowledge about individual tracks, the part RL can act on, is the smaller part. On KION the title alone predicts almost as well as the user (0.66 against 0.67): there the reward is to a large extent a property of the title, and a policy can raise it by choosing titles.
- *Where it errs.* The KION models overestimate the watched share of the least popular fifth of the catalogue by 0.13–0.18, against 0.02 for the most popular titles. On Yambda the mean error stays within 0.03 in every decile of popularity and of duration.
- *Recommended against organic events.* On Yambda the model is more accurate on events served by a recommender (AUC 0.895, MAE 0.198) than on events the user chose (AUC 0.836, MAE 0.270). RL asks the model about recommendations.

5.2 Fine-tuning at three operating points

Table 4 shows the main RL run of three settings. They end in three different places: a collapse, a large confirmed gain, and almost no change.

A weak anchor: the policy maximises the reward and stops recommending. On Yambda with REINFORCE and $\beta = 0.1$, the predicted share rises and nearly everything else is lost. Four observations explain how.

What it moved to. The fifteen tracks with the most impressions have between 136 and 2,294 plays in the training period (the most popular track has 39,487), last between 90 and 165 seconds, and were finished

Table 3: Prediction of the consumed share on the events of the test period, each from the events before it (runs F and G). AUC is for the event "consumed at least half". The reward models of run A, which have no return and choice heads, reach AUC 0.867 and 0.865.

Predictor of the consumed share	**MAE**	**RMSE**	**Cross-entropy**	**AUC**
Yambda-50M, 1.08 M test events				
Training reward model	0.235	0.327	0.432	0.867
Held-out reward model	0.234	0.327	0.433	0.867
Mean over the user's previous events	0.289	0.370	0.510	0.798
Mean share of the item	0.393	0.429	0.623	0.607
Global mean	0.409	0.434	0.635	0.500
KION, 184,209 test events				
Training reward model	0.288	0.349	0.611	0.753
Held-out reward model	0.289	0.349	0.610	0.755
Mean over the user's previous events	0.331	0.384	0.670	0.673
Mean share of the item	0.334	0.379	0.657	0.656
Global mean	0.370	0.399	0.689	0.500

Table 4: The main RL run of three settings against the pretrained recommender, on generated lists for test users: run A (4,000 steps), run F and run G (2,000 iterations each). *Share* and *time* are the consumed share and the consumption time that a reward model (RM) predicts for the lists; the share for another user and the personal part are predicted by the held-out model (Section 3.7); the KL divergence is per list position. Run A did not measure the empty cells.

	Yambda, REINFORCE $\beta = 0.1$			**Yambda, PPO $\beta = 0.3$**			**KION, PPO $\beta = 0.3$**		
	Base	**RL**	**Change**	**Base**	**RL**	**Change**	**Base**	**RL**	**Change**
NDCG@10, all	0.1175	0.0125	−89%	0.1165	0.1198	+2.9%	0.1173	0.1042	−11.1%
NDCG@10, liked	0.1010	0.0123	−88%	0.1012	0.1040	+2.8%	0.1214	0.1238	+2.0%
Recall@10, skipped	0.0737	0.0059	−92%	0.0714	0.0730	+2.2%	0.1382	0.1108	−19.8%
NDCG@10, above usual				0.1005	0.1034	+2.9%	0.1261	0.1145	−9.2%
Recall@10, below usual				0.0721	0.0734	+1.8%	0.1300	0.1137	−12.5%
Share, training RM	0.580	0.811	+40%	0.600	0.617	+2.8%	0.438	0.543	+23.9%
Share, held-out RM	0.613	0.713	+16%	0.595	0.606	+1.8%	0.459	0.565	+23.1%
Share for another user				0.503	0.510	+1.5%	0.446	0.550	+23.4%
Personal part				0.092	0.096		0.013	0.014	
Time, held-out RM (s)	123.1	93.2	−24%	119.2	117.7	−1.3%	6,464	5,911	−8.5%
Catalogue coverage	0.523	0.050		0.519	0.507		0.495	0.437	
Top-item share	0.023	0.821		0.026	0.038		0.566	0.544	
Pair overlap	0.001	0.466		0.0014	0.0021		0.209	0.261	
Mean item duration	203 s	130 s		203 s	197 s		5.7 h	3.3 h	
KL to π_0 (nats)		2.8			0.04			0.10	

to 85–97% by their listeners, against 65% overall. The most shown of them is in the lists of 6,235 of 7,593 users. These are niche tracks with a self-selected audience: those who found them like them, and the reward model extends this to everybody. The policy also abandons tracks the user already knows: the share of recommendations from the history falls from 51% to 4%.

Part of the gain is earned on the model's errors. The training reward model credits the new lists with +40%, the held-out one with +16%. The held-out estimate stops rising after about 750 steps while the training estimate keeps rising to the end (Figure 2). Everything gained after that point is seen only by the model being optimised.

The log does not confirm the gain. Relevance falls as much on liked tracks as on skipped ones. The policy has not learned to avoid what users skip; it has left the region the log says anything about.

Held-out accuracy did not protect the reward model. The model is accurate and calibrated on events that users produced. RL asks it about user–track pairs that never occurred, and there its idea of the item on average takes over.

An anchor that holds: a large and confirmed gain on KION. With PPO and $\beta = 0.3$ the predicted share rises by 23%: the held-out model reports +23.1% where the training model reports +23.9%, and in run C both estimates rise together and flatten after about 500 iterations (Figure 13 in the appendix). By the

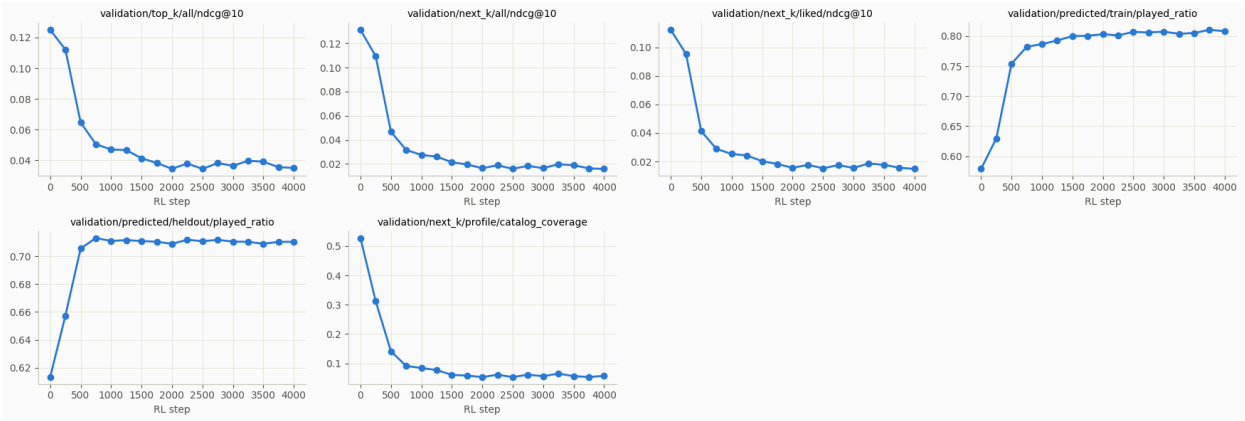


Figure 2: Greedy lists for validation users during the weakly anchored run on Yambda (REINFORCE, $\beta = 0.1$, run A). Top row: NDCG@10 of ranked lists, of generated lists, of generated lists on liked items, and the share predicted by the training reward model. Bottom row: the share predicted by the held-out reward model, and catalogue coverage. Relevance is lost within the first 500 steps; the held-out reward flattens early while the training reward keeps rising.

split into liked and skipped titles the log agrees. NDCG@10 on titles the user went on to watch at least half rises by 2.0%, with an interval for the change of [+0.0011, +0.0039], recall of abandoned titles falls by a fifth ([−0.0294, −0.0253]) and overall NDCG@10 by 11.1% ([−0.0142, −0.0120]). The policy stays close to the pretrained recommender: the KL divergence settles near 0.10 nats per position, against 2.8 in the collapsed run. Three runs of this configuration agree to within a point on every one of these changes (Table 16).

Two rows of Table 4 qualify this picture, and Section 5.4 and Section 5.5 take them up. The second split does not show a policy that keeps what this user likes: relevance falls by 9.2% on titles the user watched more than is usual for the title and by 12.5% on the others. And the same lists shown to other users are predicted to earn almost exactly the same gain.

The same weight on Yambda under PPO: almost nothing. The changes are small: +1.8% of reward by the held-out model ([+0.0096, +0.0119] in absolute terms) and +2.9% of NDCG@10 ([+0.0015, +0.0051]). Relevance rises on liked tracks; the change in recall of skipped tracks is inside its interval ([−0.0010, +0.0041]). The policy has not learned to avoid skips. It has shifted slightly towards popular tracks, which are more often in any continuation: the mean popularity of a recommended track rises by 9%, and the fifteen largest gains in impressions go mostly to tracks from the top of the catalogue, by at most 107 of 7,593 lists. The KL divergence reaches only 0.04 nats per position. In run D the value model explained about 80% of the variance of returns here, against about half on KION in run C: on Yambda the reward is mostly determined by the state of the user, not by the track chosen, which leaves the policy little to act on.

How PPO behaved. In runs C and D the share of actions beyond the clipping threshold was zero throughout on KION and 0.01% on Yambda: the small learning rate and the anchor kept every update inside the threshold without clipping. In effect this is an actor–critic method with generalised advantage estimation and four passes over each batch.

5.3 The anchor decides, and its weight does not transfer

Table 5 compares the three settings across anchor weights and Figure 3 shows the sweeps as trade-off curves; the full sweeps are in Appendix B.

Without an anchor the policy always collapses. On Yambda nearly all users receive the same ten tracks under both estimators (pair overlap 1.00; coverage 0.0002 of 50,000 tracks under REINFORCE). On KION fewer than a hundred titles are recommended at all, and two users share 99% of their lists. PPO's clipped objective is no protection: without the anchor PPO collapses exactly as REINFORCE does.

On KION the trade-off is smooth. Every loosening of the anchor buys predicted reward with relevance, and no step is a cliff. From weight 1 down to 0.2 the reward gain grows from 8% to 27% while NDCG@10 on liked titles stays within 4% of the pretrained recommender, mostly above it; recall of abandoned titles falls by 2% to 27%. Below 0.2 liked titles go as well: at 0.1 the reward is 44% higher, NDCG@10 on liked titles 24% lower and overall NDCG@10 39% lower. The two reward models agree at every weight except

Table 5: Change relative to the pretrained recommender in the anchor sweeps: run A (2,000 steps), run F and run G (1,000 iterations each). NDCG@10 on all future items of generated lists; reward is the share predicted by the held-out reward model; KL is the divergence from π_0 the run ended at, in nats per list position. The KION sweep has no runs at 0.01 and 0.03.

	Yambda, REINFORCE		**Yambda, PPO**			**KION, PPO**		
β	**NDCG@10**	**Reward**	**NDCG@10**	**Reward**	**KL**	**NDCG@10**	**Reward**	**KL**
0	−99.4%	+19%	−99.2%	+19%	7.64	−99.3%	+54%	6.45
0.01	−99.4%	+19%	−99.7%	+20%	5.77			
0.03	−99.5%	+14%	−98.6%	+27%	3.86			
0.1	−90%	+16%	−0.1%	+3.6%	0.16	−39%	+44%	0.56
0.3	−1.7%	+3.3%	+0.9%	+1.5%	0.03	−11%	+23%	0.09
1	+0.7%	+1.0%	+0.4%	+0.6%	0.01	−1.8%	+7.7%	0.02

zero, where the training model reports 0.91 and the held-out model 0.71. Exploitation of the reward model appears only at the edge.

On Yambda the transition is abrupt under both estimators, in different places. With REINFORCE the policy degenerates completely below 0.1, barely moves at 1 and above, and the interesting behaviour lies between 0.1 and 0.3, a range the grid stepped over. At 0.3 the result is what RL is meant to deliver, in small: NDCG@10 falls by 1.7%, NDCG@10 on liked tracks by 1.2% and recall of skipped tracks by 6.2%, and the predicted share rises by 5.3% according to the training model and 3.3% according to the held-out one. With PPO the same thing happens a factor of three lower. At $\beta = 0.1$, where REINFORCE lost 90% of NDCG@10, PPO loses nothing and gains 3.6% of reward; at 0.03 it has collapsed, with a pair overlap of 0.71 and a KL divergence that jumps from 0.16 to 3.9 nats. At 0.3 PPO moves the reward about half as much as REINFORCE did. Where the cliff lies is therefore a property of the estimator; that there is a cliff on Yambda and none on KION is a property of the data. We do not have a tested explanation for the second point. One candidate is that on Yambda the predicted share depends mostly on the user, so a policy has little to gain inside the region the log covers and the first large gains are the ones the training model alone reports: at 0.03 it reports 0.93 where the held-out model reports 0.75.

Why the weight does not transfer. The most likely explanation is advantage normalisation. In both algorithms advantages are scaled to unit variance, so the size of the policy-gradient term is fixed and the anchor weight is measured against it. What matters is then how much of the normalised advantage a single action can change. With the leave-one-out baseline the contribution of the user is subtracted exactly, and the advantage consists entirely of differences between actions. With PPO the baseline comes from a value model that explains 80% of the variance on Yambda; the remaining 20% is a mixture of differences between actions and errors of the value model, the useful signal takes a smaller part of the advantage, and the same weight holds the policy more firmly.

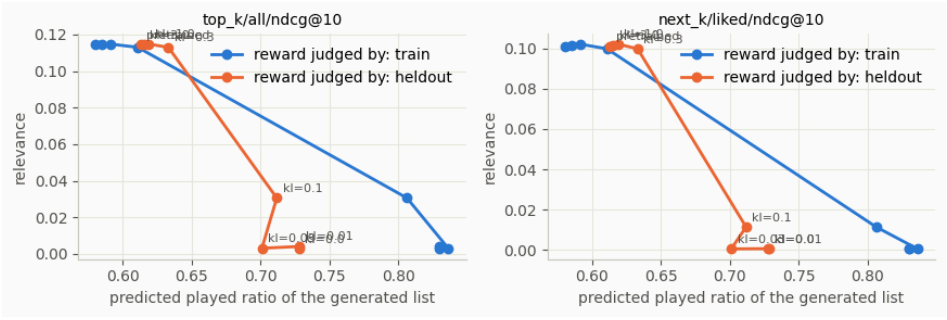
The credit ablations of run G point the same way (Table 13 in Section 5.9). With $\gamma = 0$ the advantage of an action contains only its own reward and nothing of the nine other items of the list. At the same $\beta = 0.3$ the policy then ends 2.5 nats from the pretrained recommender where the full return ends at 0.09, and loses half of its NDCG@10 where the full return loses a tenth. This remains a hypothesis for the difference between the two estimators: the learning rate (three times lower) and the number of lists per step changed together with the estimator, and the direct test, PPO with the leave-one-out baseline at the same weights, has not been run.

The practical consequence is that an anchor weight cannot be carried from one setting to another, and that runs are better placed by the KL divergence they ended at than by the weight that produced it.

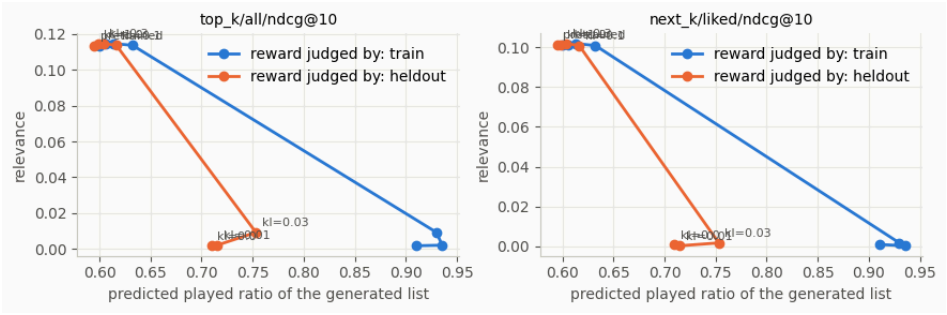
5.4 Every reward is gamed through the attribute it correlates with

The policy follows whichever property of items the reward correlates with (reward variants are tabulated in Appendix C).

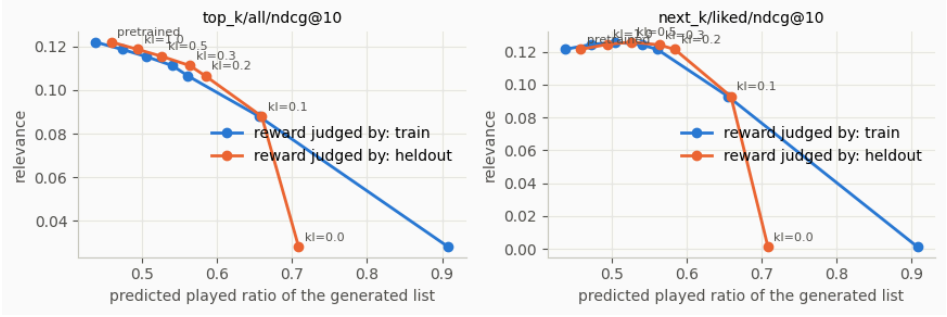
Consumed share. Under a weak anchor on Yambda the policy moves to short obscure tracks (mean duration from 203 to 130 seconds). On KION, with an anchor that holds, it moves from series to films. The titles that gain the most impressions are films of 85 to 130 minutes with a mean watched share of 0.54 to 0.76; the largest gainer goes from the lists of 8,388 users to 26,034 of 59,997. The largest losers are series: one with 17 hours of content and a mean share of 0.08 falls from 33,948 lists to 10,277, another from



(a) Yambda, REINFORCE (run A)



(b) Yambda, PPO (run F)



(c) KION, PPO (run G)

Figure 3: Relevance against predicted reward in three anchor sweeps; each point is one anchor weight. Left panels: NDCG@10 of ranked lists on all future items. Right panels: NDCG@10 of generated lists on liked items. The horizontal axis is the mean predicted share of the generated list according to the training reward model (blue) and the held-out one (orange); the horizontal distance between the curves is reward that only the optimised model reports.

16,278 to 1,542. The mean duration of a recommended title falls from 5.7 hours to 3.3. Two niche films with 953 and 175 viewers in the training period also rise, from 7,308 and 6,102 lists to 21,037 and 16,960: the Yambda pattern in a small dose.

This is what the KION gain of Section 5.2 consists of. Because the watched share of a series is taken of the whole series, a series is rarely "finished" by the one-half rule and scores low for everybody. The reward gain and the divergence between liked and skipped relevance largely restate one fact, that the policy now prefers films. The second split makes this visible. A film that this user watched to the end counts as liked, but it counts as watched above usual only if the user watched more of it than its viewers do on average, and by that measure relevance falls on both sides, by 9.2% and 12.5% at $\beta = 0.3$ and by 37% and 45% at $\beta = 0.1$ (Table 20). The predicted watch *time* falls by 8.5%. The reward model is not wrong here; the reward asks for the wrong thing. Whether a finished film is worth more than a started series is a product question that the consumed share answers by construction.

Time. The time variant was meant to remove the preference for short items. It overshoots in the other direction on both datasets. On Yambda at $\beta = 0.3$ (run D) the mean duration of recommended tracks rises from 203 to 710 seconds, the predicted listening time triples and NDCG@10 falls by 22%. On KION (run C) the mean duration of a recommended title rises from 5.7 hours to 64, the predicted watch time grows sixfold, and the predicted share, the predicted continuation and a quarter of the relevance are lost.

Table 6: Whose reward the gain is, on KION (run G). The share is predicted by the held-out reward model for the lists shown to their own users and to other users; the gain is the change of the former against the pretrained recommender; its personal share is the part that other users would not give. Re-ranking by the item mean uses no reward model.

Lists of ten	**NDCG@10, all**	**NDCG@10, liked**	**Share, own user**	**Share, other users**	**Gain**	**Personal share of gain**
Pretrained recommender	0.1173	0.1214	0.459	0.446		
RL under the anchor sweep						
$\beta = 1$	0.1152	0.1244	0.494	0.480	+0.035	3.8%
$\beta = 0.5$	0.1116	0.1258	0.526	0.512	+0.068	2.7%
$\beta = 0.3$ (main run)	0.1042	0.1238	0.565	0.550	+0.106	1.2%
$\beta = 0.2$	0.0995	0.1213	0.584	0.570	+0.126	1.0%
$\beta = 0.1$	0.0719	0.0927	0.659	0.650	+0.201	−1.7%
$\beta = 0$	0.0008	0.0012	0.709	0.709	+0.250	−5.1%
Re-ranking by the reward model						
$w = 1$	0.1199	0.1336	0.503	0.485	+0.044	12.3%
$w = 0.3$	0.1118	0.1343	0.554	0.532	+0.095	8.9%
$w = 0.1$	0.0943	0.1235	0.610	0.586	+0.151	7.3%
$w = 0$	0.0566	0.0780	0.639	0.612	+0.181	7.9%
Re-ranking by the mean share of the item, no reward model						
$w = 1$	0.1189	0.1319	0.502	0.486	+0.043	6.0%
$w = 0.3$	0.1088	0.1295	0.555	0.541	+0.096	1.2%
$w = 0.1$	0.0841	0.1078	0.616	0.605	+0.157	−1.4%
$w = 0$	0.0332	0.0433	0.631	0.625	+0.172	−4.1%

Session continuation. On Yambda a reward for continuing the session moves nothing (predicted continuation 0.9371 against 0.9372 in run D): with 95% of events continuing the session, the signal has nowhere to go. The retention signals are examined in Section 5.7.

5.5 Whose reward is it: the gain is not personal

On KION other users would give the same lists the same gain. Table 6 and Figure 4 split every gain into the part the held-out model predicts for the same lists shown to other users and the rest. For the main RL run the predicted share rises by 0.1058 for the users the lists were generated for and by 0.1045 for other users: 1.2% of the gain is personal. The personal part of the predicted share was small to begin with, 0.013 of 0.459, and it moves to 0.014 ([+0.0009, +0.0016]). Along the anchor sweep the personal share of the gain never exceeds 4%, and below $\beta = 0.2$ it is negative: the policy gives up what it knew about the user.

A reward model is not needed to reach the same frontier. Re-ranking the candidates of the pretrained recommender by the mean share of an item in the training period, a single number per item, reproduces the RL frontier. At $w = 0.3$ it reaches a predicted share of 0.555 with NDCG@10 of 0.109 overall and 0.130 on liked titles; the main RL run has 0.565 with 0.104 and 0.124. Its gain is as impersonal as that of RL. What fine-tuning against the reward model has learned on KION is, to a first approximation, this table of item means.

Re-ranking with the reward model is the method that uses the model's knowledge of the user: 7% to 12% of its gain is personal, and at equal reward it is the most relevant of the three (Figure 4). On Yambda the difference is larger (Table 25 in the appendix). There the lists of the pretrained recommender are already predicted to be worth 0.092 more to their own users than to others. Re-ranking by the reward model raises the share by up to 0.043, and 56% to 60% of that is personal. RL raises it by 0.011 at $\beta = 0.3$ with 30% personal and by 0.021 at $\beta = 0.1$ with none, and the collapsed policies lose the personal part altogether: their lists are predicted to earn exactly as much from any user.

Paying only for the personal part. Table 7 shows fine-tuning on the two rewards of Section 3.5 that subtract a baseline. The personal share behaves as its construction says. With no anchor at all the lists of different users stay apart: the pair overlap is 0.29 on KION and 0.015 on Yambda, where the plain share ends at 0.99 and 1.00. On Yambda at $\beta = 0.03$, a weight at which the plain share has collapsed, the personal share leaves relevance and coverage where they were. What it buys is small. On KION the personal part rises from 0.013 to 0.018 at $\beta = 0.3$ and to 0.028 with no anchor, and in the second case the predicted share

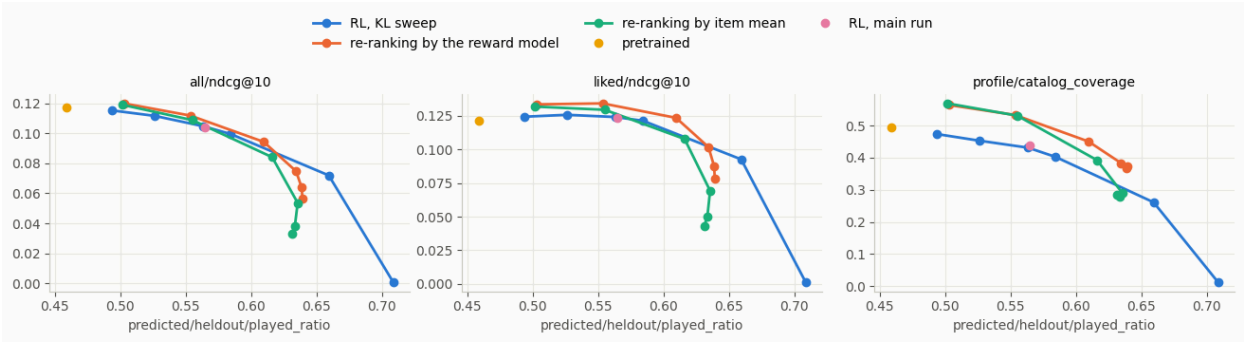


Figure 4: Relevance and coverage against the share predicted by the held-out reward model on KION (run G): RL under the anchor sweep (blue), re-ranking by the reward model (orange) and re-ranking by the mean share of the item, which uses no reward model (green). The three curves nearly coincide up to a share of about 0.6.

itself falls by a fifth and NDCG@10 by 36%; on Yambda it rises from 0.092 to 0.105 and 0.112. Re-ranking the candidates by the same personal reward reaches a personal part of about 0.07 on KION (Figure 11 in the appendix), far more than fine-tuning does. There is little that is personal in the predicted share for a policy to find, and re-ranking finds more of it.

The share above the item mean is not a safe substitute. On Yambda it collapses at the same weight, and it is the clearest case of exploitation in the study: the training reward model reports a share of 0.62 for lists that the held-out model scores at 0.45, below the 0.595 of the pretrained recommender. A reward that pays for the model's prediction exceeding the item's average pays for the model's errors.

5.6 Relevance inside the reward: logged relevance and the choice head

If the reward itself asked for relevance, the anchor might not be needed. This section tests the two mechanisms of Section 3.5.

On KION logged relevance alone keeps relevance with no anchor at all. NDCG@10 changes by +0.6% and NDCG@10 on liked titles by +5.4% (Table 8). There is no notable gain in NDCG, as in [8], where such fine-tuning matched the NDCG of supervised training. The lists, however, contract onto popular titles: coverage falls from 0.50 to 0.10, the pair overlap rises from 0.21 to 0.52, and the most popular 1% of titles now receive 91% of recommendations instead of 57%. Relevance metrics do not see this.

Table 7: Rewards measured against a baseline, 1,000 PPO iterations each. The shares are predicted by the held-out reward model for the lists shown to their own users and to other users; the personal part is their difference.

Reward	β	NDCG@10, all	NDCG@10, liked	Share, own	Share, others	Personal part	Coverage	Pair overlap
KION (run G)								
Pretrained recommender		0.1173	0.1214	0.459	0.446	0.013	0.495	0.209
Share	0.3	0.1047	0.1241	0.564	0.549	0.014	0.431	0.265
	0	0.0008	0.0012	0.709	0.709	0.000	0.012	0.989
Personal share	0.3	0.1165	0.1211	0.461	0.443	0.018	0.477	0.208
	0.1	0.1149	0.1200	0.457	0.436	0.022	0.437	0.220
	0	0.0745	0.0752	0.364	0.337	0.028	0.310	0.294
Share above the item mean	0.3	0.1094	0.1138	0.445	0.425	0.020	0.396	0.249
	0.1	0.0724	0.0797	0.489	0.476	0.013	0.193	0.400
	0	0.0380	0.0344	0.457	0.456	0.001	0.004	0.784
Yambda-50M (run F)								
Pretrained recommender		0.1165	0.1012	0.595	0.503	0.092	0.519	0.0014
Share	0.03	0.0016	0.0018	0.753	0.750	0.003	0.009	0.706
	0	0.0009	0.0010	0.710	0.710	0.000	0.0005	1.000
Personal share	0.03	0.1160	0.1006	0.604	0.499	0.105	0.502	0.0017
	0	0.0715	0.0640	0.603	0.490	0.112	0.267	0.015
Share above the item mean	0.03	0.0039	0.0034	0.448	0.426	0.022	0.019	0.762
	0	0.0005	0.0007	0.453	0.448	0.005	0.001	0.945

Table 8: Rewards with a logged-relevance term on KION (run G), 1,000 PPO iterations each. The rows with the plain share are the anchor sweep of the same run. *Logged rel.* is the logged-relevance term with the weight shown; KL is the divergence from π_0 in nats per list position.

Reward	β	NDCG@10, all	NDCG@10, liked	Recall@10, skipped	Share (held-out)	Coverage	Pair overlap	KL
Pretrained recommender		0.1173	0.1214	0.1382	0.459	0.495	0.209	
Share $\hat{\rho}$	0.3	0.1047	0.1241	0.1112	0.564	0.431	0.265	0.09
	0.2	0.0995	0.1213	0.1005	0.584	0.403	0.282	0.14
	0.1	0.0719	0.0927	0.0623	0.659	0.261	0.332	0.56
	0	0.0008	0.0012	0.0005	0.709	0.012	0.989	6.45
Logged relevance only	0	0.1180	0.1279	0.1417	0.474	0.097	0.519	1.24
Share + 10 × logged rel.	0.3	0.1191	0.1305	0.1395	0.506	0.423	0.281	0.08
	0.1	0.1146	0.1318	0.1314	0.550	0.340	0.334	0.29
	0.03	0.1056	0.1295	0.1047	0.594	0.209	0.426	0.78
	0	0.0975	0.1231	0.0852	0.639	0.101	0.567	1.83
Share + 3 × logged rel.	0.3	0.1123	0.1285	0.1270	0.544	0.420	0.271	0.08
	0.1	0.1001	0.1247	0.0968	0.605	0.317	0.356	0.34
	0.03	0.0737	0.0945	0.0612	0.704	0.143	0.360	1.19
	0	0.0821	0.1070	0.0676	0.686	0.088	0.582	1.95

Added to the predicted share, it keeps relevance better than the anchor, and not variety. With weight 10 and no anchor the predicted share rises by 39% while NDCG@10 on liked titles is unchanged (+1%) and overall NDCG@10 falls by 17%. The anchor at a higher reward ($\beta = 0.1$, +44%) loses 24% on liked titles and 39% overall. In every run with logged relevance and no anchor the coverage is about 0.10, against 0.26 for the anchor at $\beta = 0.1$. The anchor preserves the whole distribution of the pretrained recommender, its variety included; the logged-relevance reward preserves only hits, and on KION the easiest way to hit is to be popular. In run E the term stops holding when its weight is 1 or below, and the policy collapses (Table 24 in the appendix).

The two constraints together improve on either alone. A weak anchor returns most of the variety, and the relevance term lets the anchor be weaker than relevance alone would allow (Figure 5). With weight 10 and $\beta = 0.3$ the reward rises by 10% and relevance rises as well, by 1.5% overall and 7.5% on liked titles, with coverage at 0.42. At $\beta = 0.1$ the reward is 20% higher, overall NDCG@10 2.3% lower, NDCG@10 on liked titles 8.6% higher, and coverage 0.34. The plain share needs to give up 11% of NDCG@10 for a comparable 23%. With weight 3 and $\beta = 0.3$ the policy ends at the same divergence from the pretrained recommender as the plain share does (0.08 against 0.09 nats) and with the same coverage (0.42 against 0.43), and it trades a reward gain of 18% for a loss of 4.3% of NDCG@10, where the plain share trades 23% for 10.7%. Against re-ranking the combination is level on relevance and behind on variety: re-ranking at $w = 0.3$ gains 21% of reward with NDCG@10 of 0.112 and a coverage of 0.53 (Table 11).

On Yambda logged relevance raises relevance by a fifth and keeps variety. The same reward with no anchor lifts NDCG@10 from 0.1165 to 0.1415, by 21%, with coverage at 0.50 against 0.52 and a pair

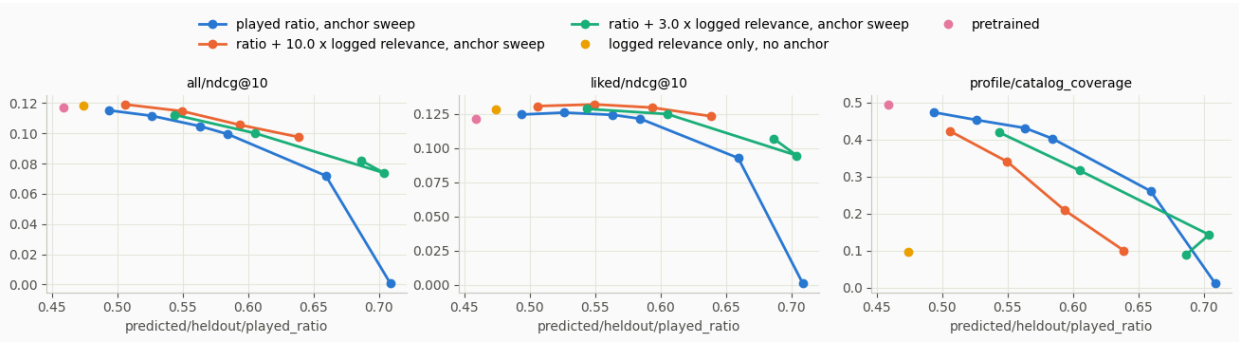


Figure 5: Relevance and coverage against the share predicted by the held-out reward model on KION (run G). Blue: the plain share under the anchor sweep. Orange and green: the share plus logged relevance with weights 10 and 3, each at four anchor weights from 0.3 down to 0. Yellow: logged relevance alone, no anchor. Pink: the pretrained recommender. The logged-relevance curves lie above the anchor sweep in relevance and below it in coverage.

overlap of 0.003 (Table 12). Added to the predicted share with weight 10 it reaches 0.1432 and a reward gain of 1.6%. No other method in this study moves relevance on Yambda by more than 8.1%. The difference from KION is in what is easiest to hit. On KION it is popular titles, which are the same for everyone. On Yambda it is the user's own earlier tracks: the share of recommendations taken from the history rises from 0.52 to 0.63, and these are different tracks for every user. The fine-tuned policy closes a third of the gap to the replay baseline of Section 5.1. Logged relevance improves relevance where the pretrained recommender left relevance on the table, and that it did so here says as much about next-item training on a log of repeats as about RL.

The choice head separates swaps but does not know relevance. The head does the task it was trained on. Its AUC for a logged item against a replacement is 0.70 on KION and 0.91 on Yambda for the training reward model, and 0.68 and 0.90 for the held-out one. As a relevance signal it fails, which shows in four ways.

- *Among the recommender's candidates.* The probability that the candidate the user really consumed next scores higher than the others is 0.56 for $\hat{q}$ on KION and 0.60 on Yambda; for the recommender's own log-probability it is 0.70 and 0.66 (Table 9). The recommender knows more about what suits the user.
- *On collapsed policies.* On KION a policy without an anchor and with NDCG@10 0.001 receives $\hat{q} \cdot \hat{\rho} =$ 0.50 from the held-out model against 0.34 for the pretrained recommender; $\hat{q}$ itself falls only from 0.74 to 0.69. On Yambda the collapsed policies receive 0.60 to 0.63 against 0.52, and $\hat{q}$ falls from 0.85 to 0.84. The quantity that was meant to fall when relevance is lost rises.
- *On the most relevant policy.* On KION the policy trained on logged relevance alone, which keeps NDCG@10, receives a lower $\hat{q}$ than the pretrained recommender, 0.71 against 0.74, and adding the share to that reward lowers it to 0.57–0.61.
- *Along the list.* On KION $\hat{q}$ rises from 0.73 at the first position to 0.80 at the tenth, while the probability of being in the real continuation falls from 0.088 to 0.016 (Figure 7).

The cause is probably the construction of the negatives. Replacements are drawn by popularity, so $\hat{q}$ measures how much more likely an item is for this user than on average, and a popular title scores about one half whoever watches it. On KION relevance is to a large extent popularity: popular items alone reach 0.090 of the recommender's 0.122. The head penalises exactly what is most often relevant. Its training also had not converged: on both datasets its validation AUC rose in every epoch while training was stopped on the total loss. Fine-tuning on $\hat{q} \cdot \hat{\rho}$ in run E confirms the diagnosis: at every anchor weight relevance is lower than with the plain share, and at $\beta = 0.1$ the policy loses 70% of NDCG@10 instead of 35% (Table 24).

5.7 Long-term signals: predictable, but barely controlled by the item

Predictability. The retention signals are predicted better than chance on both datasets, and better on KION. There the AUC of session continuation is 0.66 and that of return 0.70 on the validation period and 0.72 on the test period; on Yambda both are about 0.60 (Table 26 in the appendix). On Yambda the base rates leave little to predict: 95% of events continue the session and 92% are followed by a return within a day. In the KION test period the outcome of return is known for only 26 thousand of 184 thousand events, because the one-week window is as long as the period, so the validation period carries the evidence.

Leverage. Predictability does not make a signal a useful reward. For the 100 best candidates of every test user, Table 9 shows how much each signal varies across the candidates of one user, which is all a policy can use, and across users. On KION the consumed share varies as much across candidates as across users, and return four times less: 0.03 against 0.13. On Yambda nothing varies much across candidates. Return varies nine times less than across users (0.015 against 0.130), and the consumed share itself four times less (0.06 against 0.24), which is the reason a share reward moves so little there. Whether a user returns depends mainly on the user and hardly on what is shown.

Rewarding retention on KION. Table 10 and Figure 6 show fine-tuning on rewards that contain the retention signals.

A reward for return is a weak lever, as the spread predicted. It raises the predicted probability of return by 1.9 points (from 0.713 to 0.732) for 7% of NDCG@10 at $\beta = 0.3$, and by 4.7 points for 23% at $\beta = 0.1$. The ordinary share reward raises it by 1.5 points at $\beta = 0.3$ on its own: according to the reward model, those who finish come back more often. A dedicated reward adds less than half a point to that.

Table 9: Signals of the training reward model on the 100 best candidates of every test user (runs G and F). The first two columns of each block are standard deviations: across the candidates of one user, averaged over users, and of the per-user mean across users. *Relevance AUC* is the probability that the candidate the user really consumed next has a higher value than the other candidates.

	KION			Yambda-50M		
Signal	**Across candidates**	**Across users**	**Relevance AUC**	**Across candidates**	**Across users**	**Relevance AUC**
Consumed share $\hat{\rho}$	0.155	0.157	0.535	0.062	0.244	0.532
Session continuation $\hat{c}$	0.051	0.101	0.504	0.008	0.020	0.489
Return $\hat{u}$	0.030	0.126	0.545	0.015	0.130	0.511
Choice $\hat{q}$	0.142	0.117	0.560	0.154	0.102	0.604
$\hat{q} \cdot \hat{\rho}$	0.129	0.146	0.541	0.127	0.224	0.570
Log-probability of the recommender	0.700	0.154	0.701	0.774	0.785	0.656

Table 10: Retention rewards on KION (run E), 1,000 PPO iterations each. Share, continuation and return are the predictions of the held-out reward model for the generated lists. Run G repeated the return and share + return rows; no entry differs by more than 0.012.

Reward	**NDCG@10, all**	**NDCG@10, liked**	**Share $\hat{\rho}$**	**Contin. $\hat{c}$**	**Return $\hat{u}$**	**Coverage**	**Pair overlap**
Pretrained recommender	0.1173	0.1214	0.459	0.348	0.713	0.495	0.209
Anchor weight $\beta = 0.3$							
Share	0.1048	0.1239	0.562	0.343	0.728	0.434	0.260
Session	0.0999	0.1017	0.439	0.397	0.739	0.558	0.107
Return	0.1088	0.1130	0.462	0.368	0.732	0.517	0.147
Share + return	0.1093	0.1211	0.510	0.356	0.729	0.488	0.188
Share + session + return	0.1072	0.1160	0.494	0.367	0.733	0.519	0.156
Anchor weight $\beta = 0.1$							
Share	0.0767	0.0980	0.645	0.382	0.745	0.274	0.284
Session	0.0245	0.0287	0.161	0.609	0.769	0.385	0.334
Return	0.0903	0.0945	0.466	0.393	0.760	0.551	0.091
Share + return	0.0952	0.1139	0.559	0.365	0.749	0.467	0.186
Share + session + return	0.0874	0.1020	0.534	0.395	0.764	0.536	0.113

A reward for session continuation is harmful. At $\beta = 0.1$ the predicted continuation rises from 0.35 to 0.61 while the predicted share falls from 0.46 to 0.16 and the predicted watch time almost sixfold. The policy recommends titles that users abandon: after an abandoned title a person starts another one the same day, and the session "continues". On KION this signal rewards dissatisfaction.

Return in the reward makes the lists more diverse. The pair overlap falls from 0.21 to 0.15 at $\beta = 0.3$ and to 0.09 at 0.1, and coverage rises. Added to the share, return acts as a brake: at $\beta = 0.1$ the combined reward loses 19% of NDCG@10 where the plain share loses 35%, and the share rises by 22% instead of 41%.

Rewarding retention on Yambda does nothing. At $\beta = 0.1$ a reward for return leaves the predicted probability of return at 0.898 and everything else where it was (NDCG@10 0.1164 against 0.1165); added to the share it behaves like the share alone (NDCG@10 +2.5%, predicted share +2.0%, return unchanged). The signal that varies by 0.015 across candidates cannot be moved by choosing among them.

All return values here are predictions of a model trained on observations. It knows that people returned more often after such items; it does not know that the item was the cause. Offline one can test whether a signal is predictable, whether it has leverage and what optimising it does to the lists. Whether users actually return can only be shown online.

5.8 Generating several steps ahead

Lists drift away from the real continuation. The probability that a generated item is in the user's real continuation falls along the list: on Yambda from 0.165 at the first position to 0.06 at the tenth for the pretrained recommender, and on KION from 0.088 to 0.016 (after RL: from 0.081 to 0.018). On KION generation falls behind ranking as the list grows, and more so after RL: NDCG@10 of 0.1173 against 0.1220 before, and 0.1042 against 0.1104 after (Table 27 in the appendix). On Yambda generation stays ahead of ranking after RL as well (0.1198 against 0.1160).

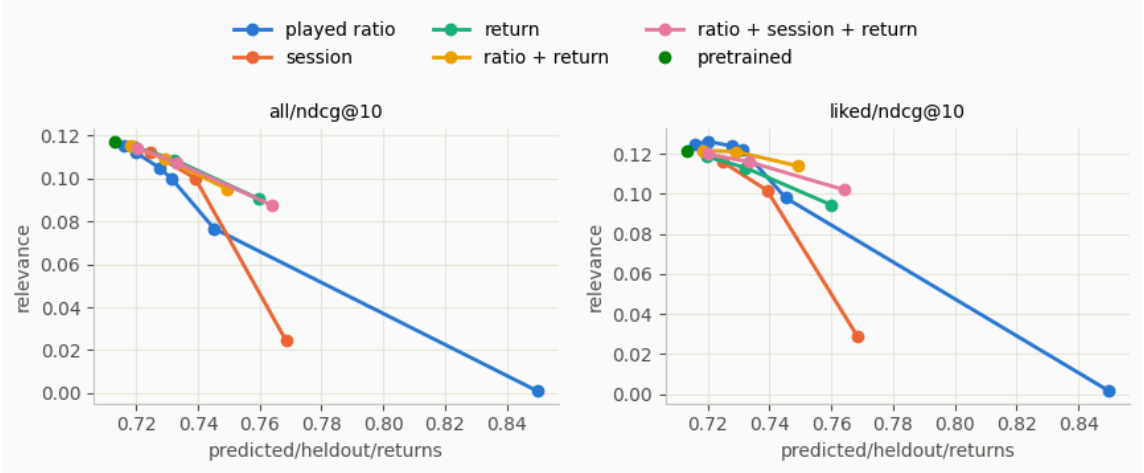


Figure 6: Relevance against the predicted probability of return on KION (run E), on all future items (left) and on liked items (right). Each line is one reward at three anchor weights. The rightmost point is the plain share with no anchor: the collapsed policy receives the highest predicted probability of return.

The reward model expects less further down the list, unless the policy shapes its context. For the pretrained recommender the predicted share falls with the position on both datasets (on Yambda from 0.67 to 0.55 in run A; on KION from 0.53 to 0.38 in run C). On KION the fine-tuned policy keeps this profile at a higher level, from 0.62 to 0.43, and its advantage shrinks from ten points at the first position to five at the tenth. The two reward models disagree by 0.05–0.06 at every position for both policies (Figure 7): RL does not take the list to where they part. For the collapsed Yambda policy of run A the picture is the opposite: the predicted share *rises* along the list, from 0.75 to more than 0.82, and the two reward models diverge more from the fourth position on.

Part of the profile is made by the simulation. A generated item has no observed feedback, but the reward model needs one to score the next item. Figure 8 scores the same lists of the fine-tuned policy under three assumptions: the model is fed its own expectation, a random outcome (finished with probability equal to the expectation, skipped otherwise), or "everything was finished". The first position depends only on the real history and is the same in all three. At the tenth position on KION the predicted share is 0.43, 0.53 or 0.68, a spread of 25 points; for the PPO policy on Yambda it is 0.56, 0.70 or 0.90, a spread of about 35 points; for the collapsed Yambda policy it is 0.82, 0.91 or 0.96. The expectation is the most pessimistic assumption in all cases, probably because of the input it creates: real consumed shares are close to 0 or to 1, and the reward model has hardly seen a history of items each consumed about halfway. Since RL was trained with this assumption, both the level of its multi-step reward and the decline along the list are partly its artefacts. The reward model has also learned that a user who has just finished several items will

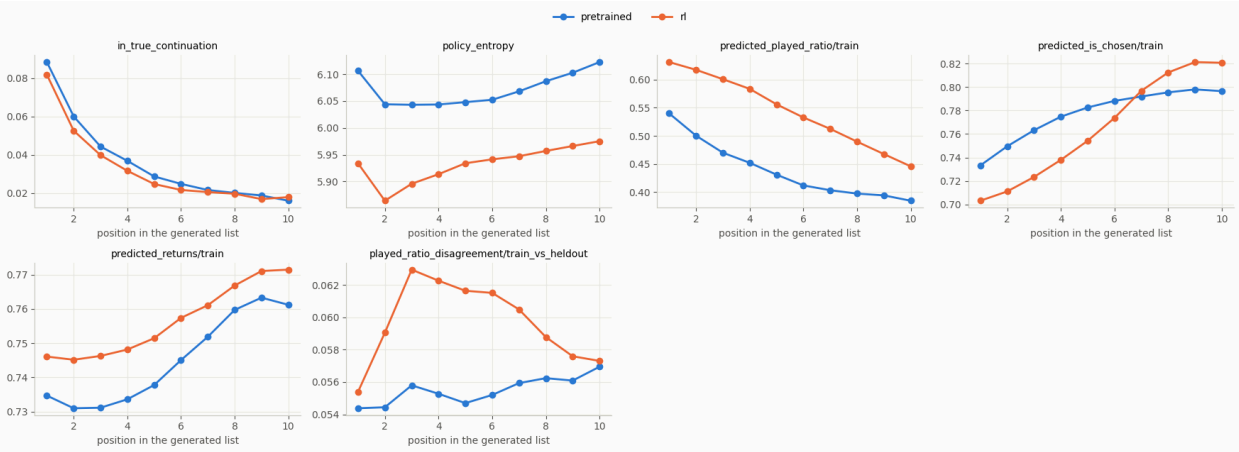
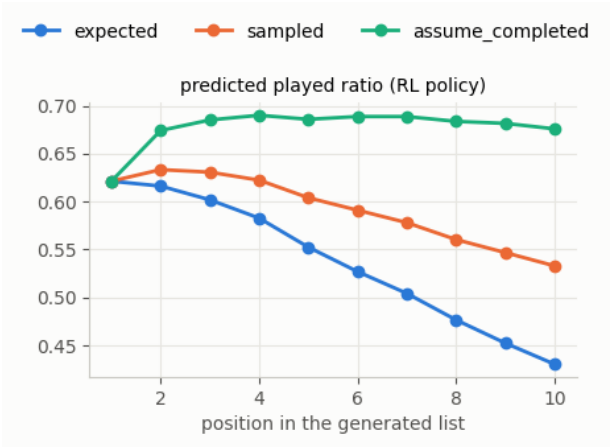
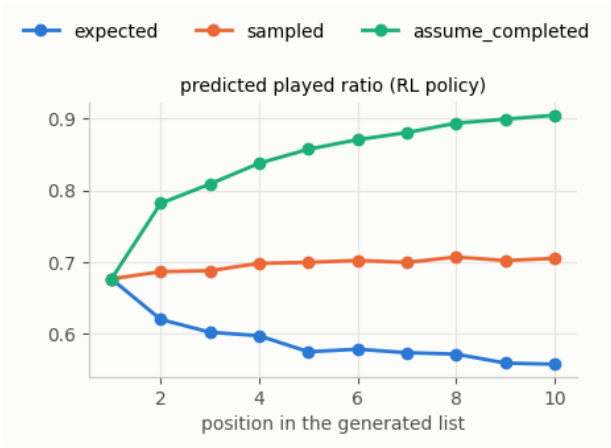


Figure 7: Generated lists by position on KION (run E): the pretrained recommender against the main RL run. Top row: probability that the item is in the user's real continuation, entropy of the policy, predicted share, and predicted choice probability. Bottom row: predicted probability of return, and the disagreement between the two reward models on the share. The choice and return signals rise along the list while real hits fall.



(a) KION, PPO (run C)



(b) Yambda, PPO (run D)

Figure 8: The share predicted by the training reward model for the lists of the fine-tuned policy under three assumptions about the feedback on the items generated earlier: the model's own expectation, a sampled outcome, and "everything was finished".

finish the next, so a policy that picks items with a high predicted share builds itself a favourable context. A multi-step predicted reward from this simulator cannot be reported without stating the assumption.

5.9 Does it take RL?

Table 11 and Figure 9 place the four alternatives of Section 3.6 next to RL on KION, and Table 12 does the same on Yambda.

Re-ranking matches RL up to a reward gain of about one third. At $w = 0.3$ re-ranking reaches a predicted share of 0.554 with NDCG@10 of 0.112 overall and 0.134 on liked titles; the RL run closest in reward (the main run, 0.565) has 0.104 and 0.124. At $w = 1$ re-ranking improves on the generated lists of the pretrained recommender on every axis at once. Up to a predicted share of about 0.62, re-ranking is at least as relevant as RL at equal reward, and it covers more of the catalogue throughout. Part of the advantage is the starting point: re-ranking starts from the ranked list, which on KION is 0.005 NDCG@10 better than the generated one, and the ranked lists of the RL policy itself reach 0.110 overall.

Liked-only fine-tuning lands on the main RL run. One epoch of supervised training on items watched at least half gives the same overall NDCG@10 (0.104), a slightly higher relevance on liked titles, a lower recall of abandoned ones and a predicted share of 0.552 against 0.565. It needs no reward model. In run C this stage together with all of re-ranking took 3.4 minutes, against 14.3 for the main RL run.

Off-policy REINFORCE at the cap of 0.5 lands there too, and keeps diversity. NDCG@10 is the same (0.103), NDCG@10 on liked titles is 1.9% above the pretrained recommender, and the reward gain is +17% against +23% for RL. Coverage and pair overlap stay where the pretrained recommender had

Table 11: RL and its alternatives on KION test users (run G). *Above* is NDCG@10 on items consumed above usual; *off-policy* is off-policy REINFORCE with the cap κ on importance weights; w is the weight of the recommender's log-probability in re-ranking. Run E gave the same numbers to within 0.002 for the methods it shares with this table.

Lists of ten	NDCG@10, all	NDCG@10, liked	NDCG@10, above	Recall@10, skipped	Share (held-out)	Coverage	Pair overlap
Pretrained, generated	0.1173	0.1214	0.1261	0.1382	0.459	0.495	0.209
RL, $\beta = 0.3$ (main run)	0.1042	0.1238	0.1145	0.1108	0.565	0.437	0.261
Liked-only fine-tuning	0.1036	0.1258	0.1134	0.1017	0.552	0.473	0.242
Off-policy, $\kappa = 0.5$	0.1032	0.1237	0.1127	0.1039	0.537	0.504	0.205
Off-policy, $\kappa = 2$	0.0901	0.1136	0.0998	0.0854	0.564	0.424	0.197
Re-ranking, $w = 1$	0.1199	0.1336	0.1314	0.1375	0.503	0.564	0.147
Re-ranking, $w = 0.3$	0.1118	0.1343	0.1242	0.1198	0.554	0.533	0.168
Re-ranking, $w = 0.1$	0.0943	0.1235	0.1079	0.0881	0.610	0.451	0.197
Re-ranking, $w = 0.03$	0.0747	0.1018	0.0874	0.0674	0.634	0.384	0.196
Re-ranking, $w = 0$	0.0566	0.0780	0.0662	0.0541	0.639	0.373	0.170
Best of 8 sampled lists	0.1055	0.1123	0.1130	0.1304	0.506	0.495	0.196

them (0.504 and 0.205), while RL narrows the lists (0.437 and 0.261). Raising the cap is costly: at 2, 23% of NDCG@10 is lost for the reward of the main RL run, and at 10 (run E) 51% for +26% of reward. The direction is that of [9], where raising the cap from 0.5 to 10 cost 9% of recall, but the effect is much stronger here. At no cap does the policy converge to a few items: at 10 the coverage is still 0.32.

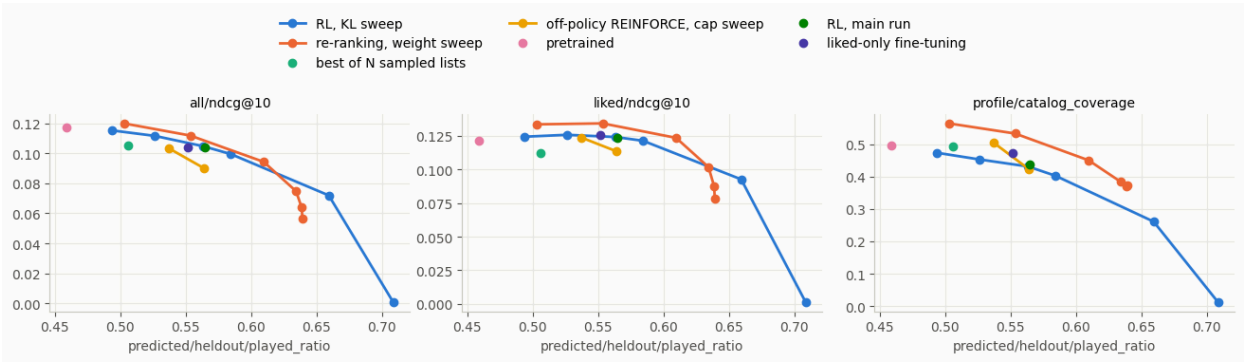
The best of several sampled lists is the weakest method. Sampling already costs relevance (0.106 with eight lists against 0.117 for the greedy list), and the reward grows slowly: +4% with two lists, +10% with eight and +15% with thirty-two (two and thirty-two in run E).

RL reaches further. Re-ranking 100 candidates cannot lift the predicted share above 0.64, and close to that limit its relevance drops sharply. RL at $\beta = 0.1$ reaches 0.659 with NDCG@10 of 0.072, above the 0.057 of re-ranking at its limit. This is the one region where RL is the better tool, and it is the region where relevance on liked titles is already a quarter lower.

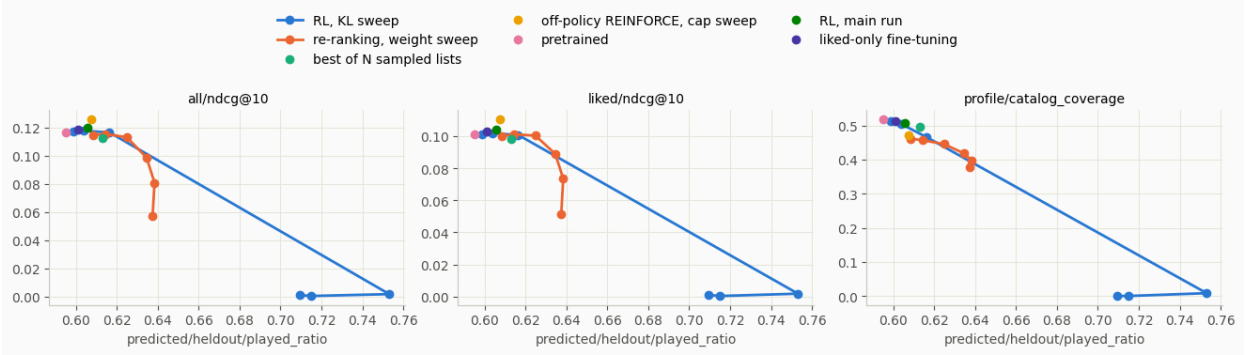
No method keeps what this user likes more than others do. By the second split every method that raises the reward loses relevance on titles the user watched above usual: RL, liked-only fine-tuning and off-policy REINFORCE by 9% to 11% at their common reward, re-ranking at $w = 0.3$ by 6% against the ranked list it starts from (0.1326). Liked-only fine-tuning shows that this is not specific to the reward model: training on "finished at least half" teaches the same preference for titles that are easy to finish.

On Yambda learning from the log is the better tool. Every method that relies on the reward model yields a reward gain of 1% to 7% (Table 12). RL at $\beta = 0.1$ and re-ranking at $w = 0.3$ are level, with +3.6% and +3.3% of reward at unchanged relevance, and liked-only fine-tuning again lands on the main RL run. Off-policy REINFORCE on logged sessions beats all of them on both axes at once: NDCG@10 rises by 8.1% and the predicted share by 2.1%. The logged-relevance rewards of Section 5.6 go further in relevance, by 20% to 23%. Both read their reward from the log and not from the model.

Does fine-tuning use the sequence? The probable reason why re-ranking keeps up with RL is the reward. The predicted share is a sum of terms for individual items, and on KION it is mostly a property of the title. Choosing items with a high predicted reward one at a time is then nearly optimal, which is



(a) KION (run G)



(b) Yambda-50M (run F)

Figure 9: Relevance and coverage against the share predicted by the held-out reward model: RL under the anchor sweep, re-ranking under the weight sweep, off-policy REINFORCE, the best of eight sampled lists, the main RL run, liked-only fine-tuning and the pretrained recommender. On Yambda the anchor sweep has two groups of points, runs that barely moved and runs that collapsed, with nothing in between.

Table 12: RL, its alternatives and the logged-relevance rewards on Yambda test users (run F). The last column is the share of recommended tracks that are in the user's history.

Lists of ten	**NDCG@10, all**	**NDCG@10, liked**	**Recall@10, skipped**	**Share (held-out)**	**Coverage**	**From history**
Pretrained recommender, generated	0.1165	0.1012	0.0714	0.595	0.519	0.520
RL, $\beta = 0.3$ (main run)	0.1198	0.1040	0.0730	0.606	0.507	0.527
RL, $\beta = 0.1$	0.1164	0.1008	0.0690	0.617	0.465	0.480
Liked-only fine-tuning	0.1183	0.1029	0.0720	0.601	0.512	0.522
Off-policy REINFORCE, $\kappa = 0.5$	0.1259	0.1107	0.0760	0.608	0.470	0.489
Re-ranking, $w = 1$	0.1145	0.0999	0.0736	0.608	0.462	0.426
Re-ranking, $w = 0.3$	0.1153	0.1012	0.0738	0.615	0.457	0.427
Re-ranking, $w = 0.1$	0.1129	0.1003	0.0718	0.625	0.445	0.416
Re-ranking, $w = 0.03$	0.0985	0.0890	0.0591	0.635	0.418	0.368
Re-ranking, $w = 0$	0.0569	0.0514	0.0324	0.638	0.377	0.258
Best of 8 sampled lists	0.1122	0.0981	0.0695	0.613	0.495	0.469
RL on rewards with logged relevance						
Logged relevance only, $\beta = 0$	0.1415	0.1249	0.0886	0.599	0.497	0.625
Share + 10 × logged rel., $\beta = 0.03$	0.1403	0.1229	0.0875	0.604	0.498	0.590
Share + 10 × logged rel., $\beta = 0$	0.1432	0.1254	0.0898	0.605	0.481	0.615

what re-ranking does. RL should win when the reward depends on the list as a whole, and this reward depends on the list only through the simulated feedback on earlier items. The credit ablations test this directly (Table 13, Figure 10).

Three things can be read from them. First, the weight of the anchor means something else once the credit changes (Section 5.3): at the same β the ablated policies end six to thirty times further from the pretrained recommender on KION, so the comparison has to be made in the plane of reward and relevance and not row by row. Second, in that plane the ablated runs on KION are not below the full-return frontier. One-item episodes at $\beta = 0.3$ reach a share of 0.678 with NDCG@10 of 0.076, above and to the right of the full return at $\beta = 0.1$ (0.659 and 0.072), and $\gamma = 0$ reaches 0.744 with 0.055, where the full-return sweep has only the collapsed run (0.709 and 0.001). The held-out model confirms these rewards. Crediting an item with the rest of the list is not what produces the gain. Third, the ablated runs pay in variety: their coverage is 0.04 to 0.06 where the full return at $\beta = 0.1$ keeps 0.26. On Yambda, at a weight where the full return has collapsed, one-item episodes keep two thirds of NDCG@10 and a coverage of 0.14 at a similar divergence from the pretrained recommender. A policy trained on one-item episodes never sees a context it has built itself, which is where the self-reinforcing part of the simulation enters (Section 5.8); this is a plausible reading of the Yambda row, not a tested one.

Table 13: Credit ablations, 1,000 PPO iterations each, against the run of the anchor sweep at the same weight. KL is the divergence from π_0 in nats per list position.

Credit	β	**NDCG@10, all**	**NDCG@10, liked**	**Share (held-out)**	**Share (training)**	**Coverage**	**KL**
KION (run G)							
Full return	0.3	0.1047	0.1241	0.564	0.540	0.431	0.09
Own reward only, $\gamma = 0$	0.3	0.0545	0.0727	0.744	0.751	0.040	2.53
One-item episodes	0.3	0.0756	0.1001	0.678	0.628	0.058	1.79
Full return	0.1	0.0719	0.0927	0.659	0.656	0.261	0.56
Own reward only, $\gamma = 0$	0.1	0.0056	0.0087	0.759	0.869	0.005	5.52
One-item episodes	0.1	0.0681	0.0898	0.656	0.603	0.017	3.54
Yambda-50M (run F)							
Full return	0.03	0.0016	0.0018	0.753	0.930	0.009	3.86
Own reward only, $\gamma = 0$	0.03	0.0031	0.0033	0.698	0.903	0.007	5.50
One-item episodes	0.03	0.0762	0.0698	0.654	0.678	0.135	3.56

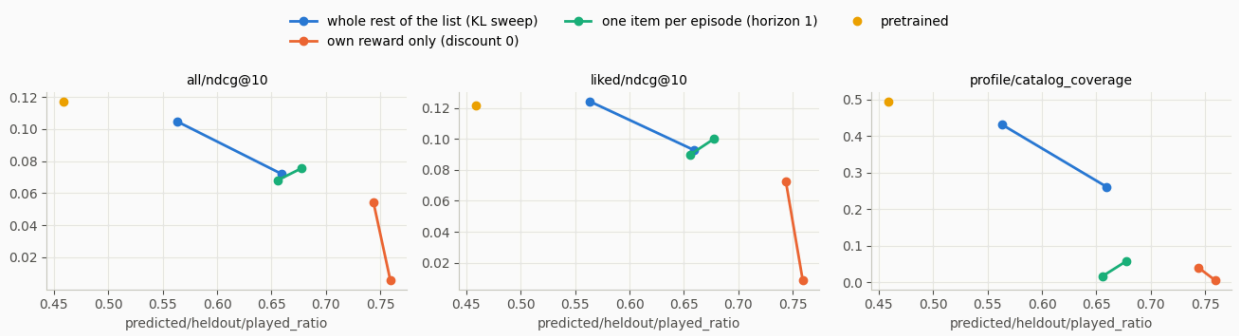


Figure 10: Relevance and coverage against the share predicted by the held-out reward model on KION (run G): the full return at $\beta = 0.3$ and 0.1 (blue), the own reward only (orange) and one-item episodes (green) at the same two weights.

6 Discussion

6.1 Relation to the two studies this work builds on

Both studies report RL that keeps relevance. Here the first main run did not keep it, the KION runs kept it at the price of 11% of overall NDCG@10, and the PPO runs on Yambda changed almost nothing. The difference from those studies is not the optimiser but where the reward comes from and on which data the policy is updated (Table 14).

Three conclusions follow. First, in a regime like theirs, with a strong anchor, the REINFORCE result on Yambda agrees with Matveev et al. almost number for number: a loss of relevance of about 2%, a loss about three times larger on interactions users did not want, and a reward gain of a few per cent. KION shows the same shape at a larger scale: −11% overall, −20% on abandoned titles, no loss on finished ones, +23% of reward. Second, both studies also lose relevance when the constraint is relaxed. Petrov and Macdonald lose 15% of NDCG@10 on MovieLens-1M and 38% on Steam-2M at their strongest popularity penalty; Matveev et al. lose 9% of recall when the cap on importance weights is raised from 0.5 to 10. What cannot happen to them is a collapse to ten items: in one case relevance to real data is inside the reward, in the other the policy never leaves logged actions. The setting of this paper, in which a learned model is the only source of reward, removes both protections. Third, the optimiser of Petrov and Macdonald is not what protects them: PPO without an anchor collapsed like REINFORCE, and with an anchor its clipping never engaged.

We tested both protections directly. The reward of [8] does keep relevance without an anchor. On KION, as its authors found on their data, it gives no notable gain in NDCG, and without a constraint on variety

Table 14: How the three settings differ. Figures for [8] are NDCG@10 on MovieLens-1M and Steam-2M across their settings; figures for [9] are from their Tables 1 and 3. Our figures are judged by the held-out reward model.

	Petrov and Macdonald [8]	**Matveev et al. [9]**	**This work**
Reward	NDCG against the real next item plus diversity or a popularity penalty	Logged like and played share of logged recommendations	Share predicted by a learned model
The policy is updated on	Its own generated lists	Logged sessions, importance weights capped at 0.5	Its own generated lists
What keeps relevance	The NDCG term inside the reward	The policy can only re-weight logged actions	The KL penalty towards the pretrained recommender
Role of a learned reward model	None	Evaluation only	Training signal
Relevance after RL	NDCG@10 from +3% to −38% depending on the weight of the second term	Recall@100 −2%; −6% on the actions with the lowest return	NDCG@10 at $\beta = 0.3$: −2% (Yambda, REINFORCE), +1% to +3% (Yambda, PPO), −11% (KION). At 0.1: −90%, 0%, −35% to −39%
Reward after RL	The target metric improves	Model-based return +2% to +4%	Predicted share at $\beta = 0.3$: +3% (Yambda, REINFORCE), +1% to +2% (Yambda, PPO), +23% (KION)

the policy contracts onto popular titles; on Yambda, where the easiest hits are the user's own repeats, it gives the largest gain in relevance of the study and no loss of variety. The method of [9] also behaves as described: at a cap of 0.5 the shift is mild and diversity is kept, relevance is lost as the cap grows, and the policy never converges to a few items; on their own dataset it is the best of the alternatives we ran. Both protections work, and each protects its own thing: the logged reward protects hits, learning on the log protects the distribution.

6.2 Four ways in which optimising the reward goes wrong

The failures observed here have distinct mechanisms, and they call for different remedies.

Exploitation of model error. The policy finds items for which the reward model errs upwards. The signature is a gap between the two reward models: +40% against +16% on Yambda under a weak anchor, 0.62 against 0.45 for the share above the item mean. The mechanism is selection: the log holds only what users chose, so for an item this user would never choose the model falls back on what it knows about the item on average, and a niche item with a devoted audience looks good for everybody. A held-out copy detects it; accuracy on logged events does not prevent it.

A misspecified reward. The model is right and the reward asks for the wrong thing. On KION the share of a series is taken of the whole series, so a policy rewarded for the share moves from series to films, the held-out model confirms the gain, and the predicted watch time falls. The time variant errs in the opposite direction, and the session reward promotes abandoned titles. A second reward model cannot detect this; inspecting what the policy moved to does, and so does a relevance split that is measured against what is usual for the item.

A reward that is not about the user. The model is right, the reward measures something real, and the cheapest way to raise it does not depend on who receives the list. On KION 99% of the gain is of this kind, and a table of item means reproduces it. Neither a second reward model nor the split into liked and skipped items exposes it; scoring the same lists for another user does. Subtracting that score from the reward removes the incentive, and shows how little is left.

A simulation artefact. The reward of late positions depends on the feedback assumed for the earlier ones, by 25 to 35 points here. Neither a second model nor relevance metrics expose it; scoring the same lists under several assumptions does.

6.3 Practical implications

Seven checks follow for anyone fine-tuning a recommender against a learned reward. (i) Fit a second reward model on other users and report every gain from both. (ii) Split relevance by the logged outcome, and measure the outcome against what is usual for the item as well as against a fixed threshold. (iii) Score every list for another user: a gain that other users would give as well is knowledge about items, and an item statistic may deliver it without any model. (iv) Do not reuse an anchor weight across estimators, rewards or datasets; report the KL divergence a run ended at. (v) Before rewarding a signal, measure its spread across the candidates of one user against its spread across users. A signal that varies only across users cannot be moved by choosing items. (vi) State the feedback assumption with any multi-step predicted reward. (vii) Compare with re-ranking by the same reward model before training anything: for an additive, item-determined reward it sets the frontier up to a substantial gain.

7 Limitations

- **One seed per configuration.** The main KION result was obtained three times and the main Yambda PPO result twice, each time with the same pretrained recommender. Intervals cover the sample of users, not the variation between training runs, and the baselines have no intervals.
- **The pretrained recommender on Yambda is below a replay heuristic.** It is the best of the trained models there, but replaying the user's history scores 60% higher in NDCG@10. Gains in relevance on Yambda are gains over this recommender.
- **The estimator explanation is a hypothesis.** The learning rate and the number of lists per step changed together with the advantage estimator; the credit ablations support it indirectly.

- **The reward study on Yambda is narrow.** Each reward ran at one or two anchor weights, the retention rewards cover return only, and the choice-weighted reward, the session reward and the time variant exist in earlier runs or on KION only.
- **The anchor grids bracket the transitions on Yambda without resolving them:** between 0.1 and 0.3 for REINFORCE, between 0.03 and 0.1 for PPO.
- **The credit ablations are compared at equal anchor weights,** which the same ablations show to be unequal constraints; no ablated run was matched to a full-return run by the divergence it reached.
- **The personal part is the reward model's own belief.** It measures how much of its prediction depends on the user, not how much of the true response does.
- **Baselines are untuned,** and GRU4Rec and BERT4Rec on Yambda were stopped by the epoch limit and not by their validation loss.
- **The choice head is one construction.** It was trained on popularity-distributed replacements and stopped before convergence; its failure speaks about this construction, not about every way of teaching a reward model relevance.
- **Return on KION is evaluated mostly on the validation period,** which also selected the stopping epoch; in the test period the outcome is known for 14% of events.
- **PPO's clipping almost never engaged,** so no run tests the part of PPO that distinguishes it.
- **Re-ranking and RL start from different lists:** re-ranking from ranked lists, RL from generated ones. On KION this favours re-ranking, on Yambda RL.
- **On KION the watched share is tied to the length of the title,** and with it the reward and the split into liked and skipped items.
- **Advantages are normalised,** which neither of the two studies we build on does; anchor weights are comparable neither with their scales nor across our settings.
- **Everything is offline.** No number here says how real users would respond to a fine-tuned policy.

8 Conclusion

We fine-tuned a generative recommender against a learned reward model in the setting with the fewest protections: the model is the only source of reward, and a KL anchor to the pretrained recommender is the only constraint. Unconstrained, the policy always collapses, and a reward model that is accurate and calibrated on logged events does not prevent it. With an anchor that holds, the gain depends on how much of the reward is a property of the item and not of the user: large and confirmed on KION, negligible on Yambda. Where it is large it is not personal. Other users would give the same lists the same reward, a table of item means reproduces the frontier, and relevance measured against what is usual for an item falls on both sides of the split. The anchor weight means different things under different advantage estimators and different credit assignment. Relevance can be required in three places, and each preserves something different: the anchor preserves the distribution of the pretrained recommender, a logged-relevance reward preserves hits and whatever is easiest to hit, and learning from logged actions preserves both for a smaller gain in reward. Retention signals can be predictable without being controllable by the choice of item, and a multi-step predicted reward is partly a property of the simulation.

For a reward that is a sum over items and mostly determined by the item, the machinery of on-policy RL buys little over re-ranking, and what it optimises is not what one would want personalised recommendation to optimise. The results that moved relevance came from the log: off-policy REINFORCE on logged sessions, and a logged-relevance term inside the reward, with a weak anchor where hits concentrate on popular items.

Several steps follow directly. A reward that combines the personal share with logged relevance would pay for hits and for what depends on the user and for nothing else; neither half has been run with the other. The normalisation hypothesis has a direct test: PPO with a leave-one-out baseline at the same anchor weights, and credit ablations matched by the divergence they reach. The gap between the pretrained recommender and the replay baseline on Yambda asks for a stronger starting point there before any conclusion about RL and relevance is drawn. A choice head with negatives drawn from the recommender's own candidates, stopped on its own quality, would test whether the reward model can be taught relevance at all. A consumed share measured relative to what is usual for the item or its type would remove the dependence on length from the reward as the second split removed it from the evaluation. Finally, every main result needs several seeds.

Code availability

The code that produced every table and figure, with the per-run reports, is in the repository `github.com/daniilantonovv/RecSysMaxxing`.

References

[1] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[2] A. V. Petrov and C. Macdonald, "Generative Sequential Recommendation with GPTRec," *arXiv preprint arXiv:2306.11114*, 2023.

[3] S. Rajput *et al.*, "Recommender Systems with Generative Retrieval," *arXiv preprint arXiv:2305.05065*, 2023.

[4] J. Zhai *et al.*, "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," *arXiv preprint arXiv:2402.17152*, 2024.

[5] J. Deng *et al.*, "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[6] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. Chi, "Top-K Off-Policy Correction for a REINFORCE Recommender System," *arXiv preprint arXiv:1812.02353*, 2018.

[7] E. Ie *et al.*, "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology," *arXiv preprint arXiv:1905.12767*, 2019.

[8] A. Petrov and C. Macdonald, "Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning," *arXiv preprint arXiv:2403.04875*, 2024.

[9] A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, and K. Khrylchenko, "Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction," *arXiv preprint arXiv:2607.02818*, 2026.

[10] T. Yu *et al.*, "MOPO: Model-based Offline Policy Optimization," *arXiv preprint arXiv:2005.13239*, 2020.

[11] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-Based Offline Reinforcement Learning," *arXiv preprint arXiv:2005.05951*, 2020.

[12] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to Trust Your Model: Model-Based Policy Optimization," *arXiv preprint arXiv:1906.08253*, 2019.

[13] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[14] T. Coste, U. Anwar, R. Kirk, and D. Krueger, "Reward Model Ensembles Help Mitigate Overoptimization," *arXiv preprint arXiv:2310.02743*, 2023.

[15] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.

[16] C. Gao *et al.*, "Alleviating Matthew Effect of Offline Reinforcement Learning in Interactive Recommendation," *arXiv preprint arXiv:2307.04571*, 2023.

[17] C. Gao *et al.*, "CIRS: Bursting Filter Bubbles by Counterfactual Interactive Recommender System," *arXiv preprint arXiv:2204.01266*, 2022.

[18] R. Jiang, S. Chiappa, T. Lattimore, A. György, and P. Kohli, "Degenerate Feedback Loops in Recommender Systems," *arXiv preprint arXiv:1902.10730*, 2019.

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019.

[20] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06939*, 2015.

[21] F. Sun *et al.*, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019.

[22] E. Botta *et al.*, "PinRec: Unified Generative Retrieval for Pinterest Recommender Systems," *arXiv preprint arXiv:2504.10507*, 2025.

[23] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Generative Slate Recommendation with Reinforcement Learning," *arXiv preprint arXiv:2301.08632*, 2023.

[24] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-Supervised Reinforcement Learning for Recommender Systems," *arXiv preprint arXiv:2006.05779*, 2020.

[25] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Supervised Advantage Actor-Critic for Recommender Systems," *arXiv preprint arXiv:2111.03474*, 2021.

[26] K. Christakopoulou *et al.*, "Reward Shaping for User Satisfaction in a REINFORCE Recommender," *arXiv preprint arXiv:2209.15166*, 2022.

[27] E. Ie *et al.*, "RecSim: A Configurable Simulation Platform for Recommender Systems," *arXiv preprint arXiv:1909.04847*, 2019.

[28] J.-C. Shi, Y. Yu, Q. Da, S.-Y. Chen, and A.-X. Zeng, "Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning," *arXiv preprint arXiv:1805.10000*, 2018.

[29] K. Zhao *et al.*, "KuaiSim: A Comprehensive Simulator for Recommender Systems," *arXiv preprint arXiv:2309.12645*, 2023.

[30] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust Region Policy Optimization," *arXiv preprint arXiv:1502.05477*, 2015.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[32] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, "Sequence Tutor: Conservative Fine-Tuning of Sequence Generation Models with KL-control," *arXiv preprint arXiv:1611.02796*, 2016.

[33] N. Jaques *et al.*, "Way Off-Policy Batch Deep Reinforcement Learning of Implicit Human Preferences in Dialog," *arXiv preprint arXiv:1907.00456*, 2019.

[34] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[35] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[36] Y. Wu, G. Tucker, and O. Nachum, "Behavior Regularized Offline Reinforcement Learning," *arXiv preprint arXiv:1911.11361*, 2019.

[37] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems," *arXiv preprint arXiv:1902.05570*, 2019.

[38] Q. Cai *et al.*, "Reinforcing User Retention in a Billion Scale Short Video Recommender System," *arXiv preprint arXiv:2302.01724*, 2023.

[39] T. A. Mann *et al.*, "Learning from Delayed Outcomes via Proxies with Applications to Recommender Systems," *arXiv preprint arXiv:1807.09387*, 2018.

[40] Y. Su *et al.*, "Long-Term Value of Exploration: Measurements, Findings and Algorithms," *arXiv preprint arXiv:2305.07764*, 2023.

[41] R. Zhan *et al.*, "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation," *arXiv preprint arXiv:2206.06003*, 2022.

[42] H. Zhao, L. Zhang, J. Xu, G. Cai, Z. Dong, and J.-R. Wen, "Uncovering User Interest from Biased and Noised Watch Time in Video Recommendation," *arXiv preprint arXiv:2308.08120*, 2023.

[43] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," *arXiv preprint arXiv:1506.03099*, 2015.

[44] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, "Sequence Level Training with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06732*, 2015.

[45] C. Guo *et al.*, "PROMISE: Process Reward Models Unlock Test-Time Scaling Laws in Generative Recommendations," *arXiv preprint arXiv:2601.04674*, 2026.

[46] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Offline Evaluation for Reinforcement Learning-based Recommendation: A Critical Issue and Some Alternatives," *arXiv preprint arXiv:2301.00993*, 2023.

[47] M. Dudik, J. Langford, and L. Li, "Doubly Robust Policy Evaluation and Learning," *arXiv preprint arXiv:1103.4601*, 2011.

[48] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches," *arXiv preprint arXiv:1907.06902*, 2019.

[49] M. Gutmann and A. Hyvärinen, "Noise-Contrastive Estimation: A New Estimation Principle for Unnormalized Statistical Models," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 2010, pp. 297–304.

[50] R. J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[51] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE Samples, Get a Baseline for Free!," in *ICLR 2019 Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

[52] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[53] A. Ploshkin *et al.*, "Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking And Retrieval," *arXiv preprint arXiv:2505.22238*, 2025.

[54] A. Petrov, I. Safilo, D. Tikhonovich, and D. Ignatov, "MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation," *arXiv preprint arXiv:2209.00325*, 2022.

[55] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *arXiv preprint arXiv:1711.05101*, 2017.

A Run configurations

Table 15: The seven completed runs. The horizon is 10 items everywhere; windows are 128 events on Yambda and 48 on KION; run B used a Tesla T4 and all others an RTX PRO 6000. REINFORCE samples 4 lists per history with learning rate $3 \cdot 10^{-5}$, PPO one list per history with 10^{-5}. Sweep runs last 2,000 steps in run A, 500 in run B and 1,000 iterations in the PPO runs.

Run	Dataset	Algorithm	Main RL run	β	β in the sweep	Beyond the anchor sweep
A	Yambda-50M	REINFORCE	4,000 × 1,024 lists	0.1	0, 0.01, 0.03, 0.1, 0.3, 1, 3	reward variants (time, session)
B	Yambda-50M, 30,000 tracks	REINFORCE	1,500 × 256	0.1	0, 0.03, 0.3, 3	reward variants
C	KION	PPO	2,000 × 512	0.3	0, 0.1, 0.2, 0.3, 0.5, 1	baselines, liked-only fine-tuning, re-ranking, intervals, reward variants
D	Yambda-50M	PPO	2,000 × 512	0.3	0, 0.1, 0.2, 0.3, 0.5, 1	as C
E	KION	PPO	2,000 × 512	0.3	0, 0.1, 0.2, 0.3, 0.5, 1	as C; return and choice heads; rewards for retention, choice-weighted and logged-relevance rewards; off-policy REINFORCE; best of n lists
F	Yambda-50M	PPO	2,000 × 512	0.3	0, 0.01, 0.03, 0.1, 0.3, 1	as E on reduced grids; replay baseline; second relevance split; personal part; personal rewards; credit ablations
G	KION	PPO	2,000 × 512	0.3	0, 0.1, 0.2, 0.3, 0.5, 1	as F on full grids

Run B. The reduced-budget run looked better at first sight: its main RL run lost 68% of NDCG@10 where run A lost 89%. It did far less work (6 epochs on half as many training windows with a smaller softmax, and about eleven times fewer generated lists in the main run), and its policy was stopped while still degrading: relevance was falling at the last measurement, and the KL divergence to the pretrained recommender was 1.7 nats and rising, where run A levelled off at 2.8. Its sweep agrees with run A where they overlap: after 500 steps the runs with weights 0 and 0.03 had collapsed (NDCG@10 0.0017 and 0.0043, pair overlap 1.00 and 0.98) and the run with 0.3 had held (NDCG@10 0.1237 against 0.1226 for its pretrained recommender, recall of skipped tracks −2.4%, predicted share +2.7% and +2.4% by the two reward models).

Table 16: Repeated PPO runs. Changes are relative to the pretrained recommender, which is the same within a dataset. The reward models of run C have no return and choice heads; those of runs E and G share one configuration.

	C	E	G	D	F
		KION		*Yambda-50M*	
Main RL run, PPO, $\beta = 0.3$					
NDCG@10, all	−11.4%	−11.4%	−11.1%	+2.2%	+2.9%
NDCG@10, liked	+1.2%	+1.3%	+2.0%	+2.0%	+2.8%
Recall@10, skipped	−19.6%	−20.2%	−19.8%	+4.4%	+2.2%
Predicted share, training RM	+23.9%	+24.0%	+23.9%	+1.8%	+2.8%
Predicted share, held-out RM	+22.8%	+23.1%	+23.1%	+1.1%	+1.8%
Catalogue coverage	0.434	0.438	0.437	0.503	0.507
Pair overlap	0.275	0.260	0.261	0.0022	0.0021
KL to π_0 (nats)	0.10		0.10	0.03	0.04
Sweep run at $\beta = 0.1$					
NDCG@10, all	−37%	−35%	−39%	+0.7%	−0.1%
Predicted share, held-out RM	+47%	+41%	+44%	+2.5%	+3.6%
Sweep run with no anchor					
Distinct items in all lists	46	81	95	about 20	about 25

Table 17: Wall-clock time by stage; not recorded for runs F and G. One PPO iteration of 512 lists takes 1.1 seconds on Yambda (catalogue 50,000, windows of 128) and 0.43 seconds on KION (catalogue 7,992, windows of 48).

Stage, minutes	**A**	**B**	**C**	**D**	**E**
Pretrained recommender	99	109	4	66	4
Reward models	6	22	1	6	4
Baselines			5	81	6
Main RL run	74	56	14	38	14
Anchor sweep	249	61	33	109	33
Alternatives to RL			3	6	16
Reward sweep	142	46	16	54	65
Relevance inside the reward					43
Everything else	1	6	3	1	3
Total	570	300	79	361	188

B Anchor sweeps

Table 18: Anchor sweep on Yambda with REINFORCE (run A): generated lists for test users after 2,000 RL steps.

Anchor weight β	**Base**	**0**	**0.01**	**0.03**	**0.1**	**0.3**	**1**	**3**
NDCG@10, all	0.1175	0.0007	0.0007	0.0006	0.0118	0.1155	0.1183	0.1176
NDCG@10, liked	0.1010	0.0007	0.0007	0.0005	0.0114	0.0998	0.1021	0.1014
Recall@10, skipped	0.0737	0.0002	0.0002	0.0001	0.0059	0.0691	0.0734	0.0734
Predicted share, training RM	0.580	0.830	0.830	0.836	0.806	0.611	0.591	0.585
Predicted share, held-out RM	0.613	0.728	0.728	0.701	0.712	0.633	0.619	0.615
Catalogue coverage	0.523	0.0002	0.0002	0.0004	0.046	0.472	0.514	0.520
Pair overlap	0.001	1.000	1.000	0.895	0.596	0.005	0.002	0.002
Mean duration (s)	203	119	119	123	125	189	199	202
Share from the history	0.505	0.005	0.005	0.003	0.034	0.479	0.505	0.507

Table 19: Anchor sweep on Yambda with PPO (run F): generated lists for test users after 1,000 iterations.

Anchor weight β	**Base**	**0**	**0.01**	**0.03**	**0.1**	**0.3**	**1**
NDCG@10, all	0.1165	0.0009	0.0003	0.0016	0.1164	0.1175	0.1170
NDCG@10, liked	0.1012	0.0010	0.0004	0.0018	0.1008	0.1018	0.1012
Recall@10, skipped	0.0714	0.0000	0.0000	0.0006	0.0690	0.0718	0.0722
NDCG@10, above usual	0.1005	0.0010	0.0004	0.0019	0.1002	0.1013	0.1006
Recall@10, below usual	0.0721	0.0001	0.0000	0.0007	0.0710	0.0724	0.0727
Predicted share, training RM	0.600	0.910	0.936	0.930	0.632	0.613	0.606
Predicted share, held-out RM	0.595	0.710	0.715	0.753	0.617	0.604	0.599
Personal part	0.092	0.000	0.001	0.003	0.092	0.095	0.094
Catalogue coverage	0.519	0.0005	0.0005	0.009	0.465	0.504	0.513
Pair overlap	0.0014	1.000	0.879	0.706	0.0043	0.0020	0.0017
Share from the history	0.520	0.003	0.001	0.006	0.480	0.514	0.519
KL to π_0 (nats)		7.64	5.77	3.86	0.16	0.03	0.01

Table 20: Anchor sweep on KION with PPO (run G): generated lists for test users after 1,000 iterations.

Anchor weight β	**Base**	**0**	**0.1**	**0.2**	**0.3**	**0.5**	**1**
NDCG@10, all	0.1173	0.0008	0.0719	0.0995	0.1047	0.1116	0.1152
NDCG@10, liked	0.1214	0.0012	0.0927	0.1213	0.1241	0.1258	0.1244
Recall@10, skipped	0.1382	0.0005	0.0623	0.1005	0.1112	0.1282	0.1351
NDCG@10, above usual	0.1261	0.0009	0.0800	0.1097	0.1147	0.1216	0.1244
Recall@10, below usual	0.1300	0.0007	0.0717	0.1069	0.1146	0.1245	0.1289
Predicted share, training RM	0.438	0.908	0.656	0.561	0.540	0.505	0.473
Predicted share, held-out RM	0.459	0.709	0.659	0.584	0.564	0.526	0.494
Personal part	0.013	0.000	0.010	0.014	0.014	0.015	0.014
Predicted time, held-out RM (s)	6,464	4,190	4,096	5,910	6,090	6,213	6,312
Catalogue coverage	0.495	0.012	0.261	0.403	0.431	0.453	0.474
Pair overlap	0.209	0.989	0.332	0.282	0.265	0.236	0.221
Mean duration (hours)	5.7	1.6	1.8	3.1	3.5	4.3	4.9
KL to π_0 (nats)		6.45	0.56	0.14	0.09	0.04	0.02

C Reward variants

In the first three tables of this section *time* multiplies the predicted share by the relative duration of the item, and *session* is the predicted probability that the session continues: another listen within 30 minutes on Yambda, another title within 7 days on KION (run C). The Yambda REINFORCE variants ran at an anchor weight that the sweep later placed on the collapsing side, so Table 21 shows in which direction each reward is gamed and is not a fair comparison of rewards.

Table 21: Reward variants on Yambda with REINFORCE (run A), 2,000 steps each at $\beta = 0.1$.

Reward	**Base**	**Share**	**Time**	**Share + session**	**Session only**
NDCG@10, all	0.1175	0.0118	0.0003	0.0105	0.0056
Predicted share (held-out)	0.613	0.712	0.545	0.710	0.634
Predicted seconds (held-out)	123	89	801	89	110
Predicted continuation (held-out)	0.937	0.941	0.869	0.941	0.941
Catalogue coverage	0.523	0.046	0.0004	0.040	0.025
Pair overlap	0.001	0.596	0.960	0.555	0.111
Mean duration (s)	203	125	1,513	125	180

Table 22: Reward variants on Yambda with PPO (run D), 1,000 iterations each at $\beta = 0.3$.

Reward	**Base**	**Time**	**Share + session**	**Session only**
NDCG@10, all	0.1165	0.0910	0.1173	0.1164
NDCG@10, liked	0.1012	0.0783	0.1015	0.1006
Predicted share (held-out)	0.610	0.553	0.614	0.610
Predicted seconds (held-out)	122	364	121	122
Predicted continuation (held-out)	0.9371	0.9023	0.9372	0.9372
Catalogue coverage	0.519	0.339	0.509	0.513
Pair overlap	0.0014	0.134	0.0020	0.0016
Mean duration (s)	203	710	200	203

Table 23: Reward variants on KION with PPO (run C), 1,000 iterations each at $\beta = 0.3$. None of them collapsed.

Reward	**Base**	**Share**	**Time**	**Share + session**	**Session only**
NDCG@10, all	0.1173	0.1042	0.0885	0.1098	0.1093
NDCG@10, liked	0.1214	0.1240	0.0901	0.1217	0.1131
Recall@10, skipped	0.1382	0.1099	0.1042	0.1281	0.1324
Predicted share (held-out)	0.444	0.544	0.307	0.487	0.440
Predicted hours (held-out)	1.77	1.60	10.51	1.62	1.61
Predicted continuation (held-out)	0.804	0.805	0.763	0.812	0.821
Catalogue coverage	0.495	0.423	0.274	0.487	0.505
Pair overlap	0.209	0.280	0.304	0.189	0.144
Mean duration (hours)	5.7	3.3	63.6	4.3	5.1

On KION, adding the 7-day session signal of run C to the share halves the shift: the share rises by 10% instead of 23%, relevance on liked titles is unchanged and overall NDCG@10 falls by 6%. The session-only reward raises the predicted probability of another viewing within a week from 0.804 to 0.821 and leaves the share where it was, at a cost of 7% of NDCG@10; it is the one variant that makes the lists less alike (pair overlap from 0.21 to 0.14).

Table 24: Rewards with their own relevance term on KION in run E, 1,000 PPO iterations each: the choice-weighted share, and logged relevance at four weights with no anchor.

Reward	β	**NDCG@10, all**	**NDCG@10, liked**	**Recall@10, skipped**	**Share (held-out)**	**Coverage**	**Pair overlap**
Pretrained recommender		0.1173	0.1214	0.1382	0.459	0.495	0.209
Share $\hat{\rho}$	0.3	0.1048	0.1239	0.1123	0.562	0.434	0.260
	0.1	0.0767	0.0980	0.0686	0.645	0.274	0.284
	0	0.0010	0.0015	0.0005	0.743	0.010	0.987
Choice-weighted share $\hat{q} \cdot \hat{\rho}$	0.3	0.0994	0.1141	0.1105	0.539	0.447	0.156
	0.1	0.0352	0.0465	0.0244	0.672	0.238	0.477
	0	0.0004	0.0006	0.0002	0.687	0.004	0.999
Logged relevance only	0	0.1180	0.1279	0.1417	0.474	0.097	0.519
Share + 10 × logged relevance	0	0.0971	0.1229	0.0843	0.640	0.103	0.577
Share + 3 × logged relevance	0	0.0749	0.0970	0.0622	0.698	0.088	0.498
Share + 1 × logged relevance	0	0.0088	0.0137	0.0035	0.718	0.013	0.964
Share + 0.3 × logged relevance	0	0.0042	0.0066	0.0020	0.714	0.012	0.984

D Further tables and figures

Table 25: Whose reward the gain is, on Yambda (run F); the columns are those of Table 6. A negative personal share means that the lists lost part of what made them worth more to their own users.

Lists of ten	**NDCG@10, all**	**NDCG@10, liked**	**Share, own user**	**Share, other users**	**Gain**	**Personal share of gain**
Pretrained recommender	0.1165	0.1012	0.595	0.503		
RL under the anchor sweep						
$\beta = 1$	0.1170	0.1012	0.599	0.505	+0.004	38%
$\beta = 0.3$ (main run)	0.1198	0.1040	0.606	0.510	+0.011	30%
$\beta = 0.1$	0.1164	0.1008	0.617	0.524	+0.021	−1%
$\beta = 0.03$	0.0016	0.0018	0.753	0.750	+0.158	−57%
$\beta = 0$	0.0009	0.0010	0.710	0.710	+0.115	−81%
Re-ranking by the reward model						
$w = 1$	0.1145	0.0999	0.608	0.508	+0.013	58%
$w = 0.3$	0.1153	0.1012	0.615	0.511	+0.020	60%
$w = 0.1$	0.1129	0.1003	0.625	0.515	+0.030	59%
$w = 0$	0.0569	0.0514	0.638	0.521	+0.043	56%
Re-ranking by the mean share of the item, no reward model						
$w = 1$	0.1139	0.0993	0.609	0.510	+0.014	44%
$w = 0.3$	0.1134	0.0994	0.616	0.518	+0.021	29%
$w = 0.1$	0.1081	0.0952	0.629	0.535	+0.034	4%
$w = 0$	0.0384	0.0335	0.634	0.556	+0.039	−35%

Table 26: Prediction of the retention signals by the training reward model (runs G and F). The held-out model differs by at most 0.004 on KION and 0.014 on Yambda.

Dataset	Signal	Period	AUC	Base rate	Events with a known outcome
KION	Session continuation	validation	0.660	0.427	187,398
	Session continuation	test	0.661	0.421	184,209
	Return	validation	0.696	0.720	187,398
	Return	test	0.719	0.702	26,004
Yambda-50M	Session continuation	validation	0.608	0.950	1,127,899
	Session continuation	test	0.602	0.950	1,081,970
	Return	validation	0.596	0.912	1,127,899
	Return	test	0.594	0.916	914,439

Table 27: Ranking against item-by-item generation on test users, before and after the main RL run. The first item is the same by construction.

NDCG at	Base, ranked	Base, generated	RL, ranked	RL, generated
KION (run G)				
1	0.0885	0.0885	0.0819	0.0819
5	0.1055	0.1044	0.0947	0.0916
10	0.1220	0.1173	0.1104	0.1042
Yambda-50M (run F)				
1	0.1653	0.1653	0.1678	0.1678
5	0.1276	0.1326	0.1294	0.1355
10	0.1136	0.1165	0.1160	0.1198

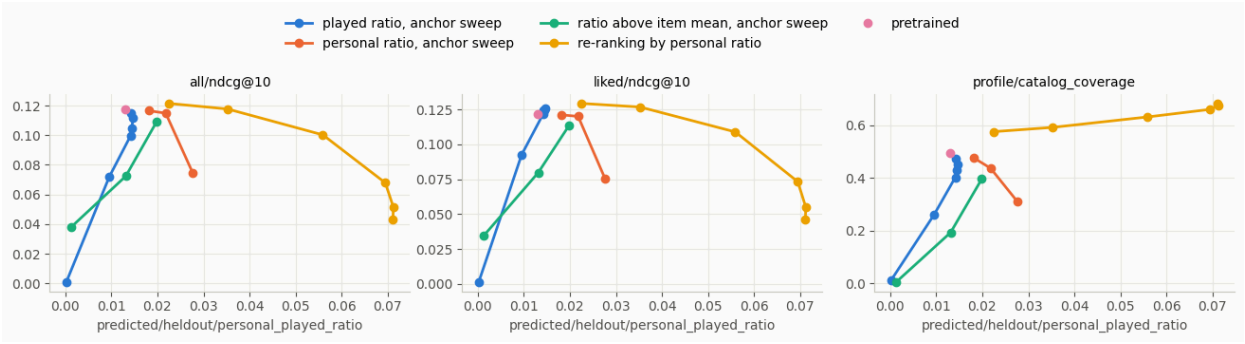
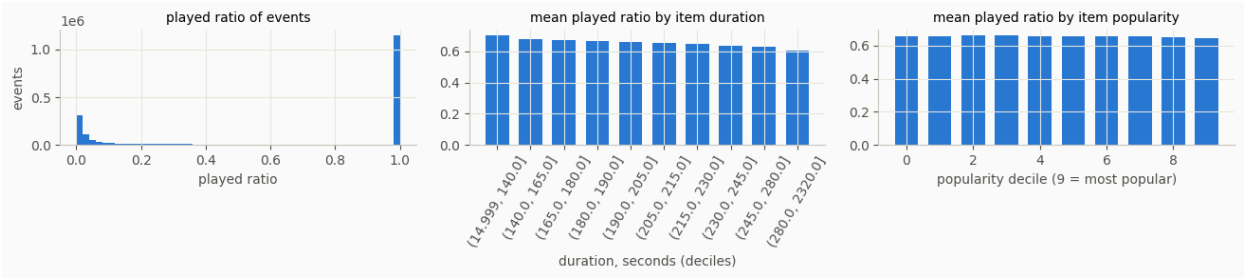
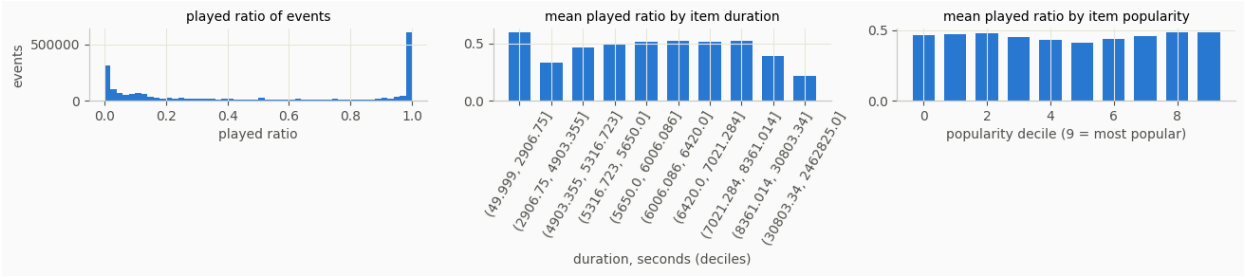


Figure 11: Relevance and coverage against the personal part of the predicted share on KION (run G): the plain share under the anchor sweep (blue), the personal share (orange) and the share above the item mean (green) at three anchor weights each, and re-ranking the candidates of the pretrained recommender by the personal share (yellow).



(a) Yambda-50M, catalogue of 50,000 tracks



(b) KION

Figure 12: The reward signal in the two processed logs. Left: distribution of the consumed share. Middle and right: mean consumed share by decile of item duration and of item popularity.

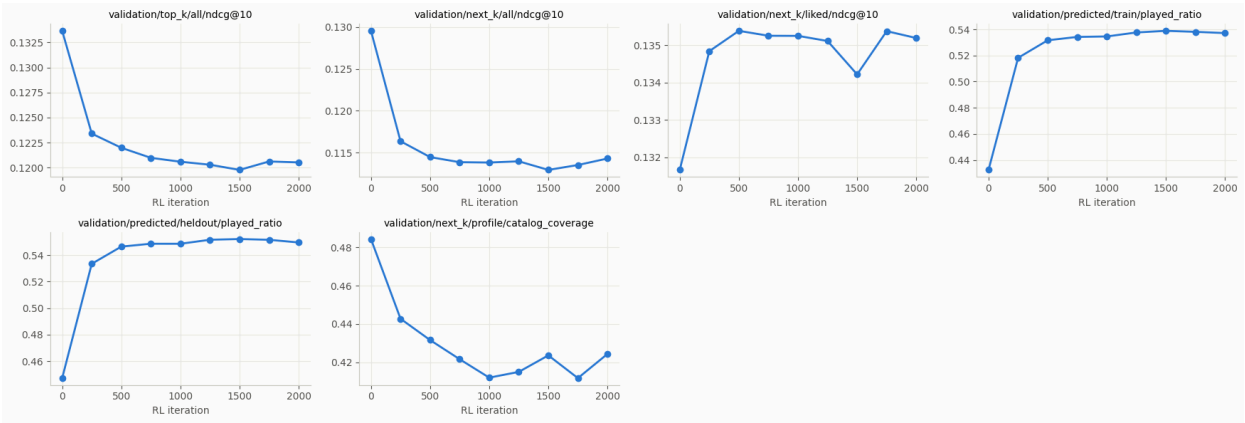


Figure 13: Greedy lists for validation users during the main RL run on KION (PPO, $\beta = 0.3$, run C). Over the first 500 iterations relevance on all titles falls and relevance on liked titles rises; the reward by both models flattens at the same time.